UNIVERSIDAD DE MURCIA

FACULTAD DE MATEMÁTICAS

Fundamentos Matemáticos del Aprendizaje
Semi-Supervisado

D. Javier Patricio Luján Romero
Mayo 2026

A mi familia por su amor incondicional desde el inicio.

Mi madre y su ternura.

Mi padre y su sacrificio.

Índice general

Abstract III

Resumen IV

1. Introducción 1

1.1. Conceptos iniciales 1

1.2. Estructura del trabajo 2

2. Introducción al Aprendizaje Semi-Supervisado 4

2.1. Problema de clasificación semi-supervisado 4

2.2. Funciones de pérdida y regularización 5

2.3. Suposiciones del aprendizaje semi-supervisado 7

3. Taxonomía de métodos de aprendizaje semi-supervisado 9

3.1. Características discriminantes 9

3.1.1. Suposición sobre la relación entre los datos etiquetados y no etiquetados 9

3.1.2. Inductivo vs transductivo 10

3.1.3. Generativo vs discriminante 10

3.1.4. Basado en el uso de los datos no etiquetados 10

3.2. Taxonomía 11

4. Métodos envolventes 14

4.1. Auto-entrenamiento 15

4.2. Métodos basados en desacuerdo 15

4.2.1. Múltiples vistas 16

4.2.2. Una única vista 16

4.2.3. Co-regularización 17

4.3. Boosting 17

5. Preprocesado no-supervisado de datos **19**
5.1. Extracción de características 19
5.2. Cluster-then-label 21
5.3. Pre-entrenamiento de modelos 22

6. Métodos intrínsecamente semi-supervisados **24**
6.1. Generativos 24
6.1.1. Mezcla de modelos 24
6.2. Discriminativos 26
6.2.1. Máximo margen 26
6.2.2. Basados en perturbaciones 28
6.3. Basados en variedades 30
6.3.1. Técnicas de regularización de la variedad 30

7. Métodos Transductivos **32**
7.1. Construcción del grafo 33
7.1.1. Construcción de la matriz de adyacencia 33
7.1.2. Asignación de pesos 33
7.2. Inferencia sobre el grafo 34
7.2.1. Mincut 35
7.2.2. Funciones armónicas 35

Bibliografía **36**

Abstract

El artículo 5.2 de la *Normativa propia de la Facultad de Matemáticas de la Universidad de Murcia para el Trabajo de Fin de Grado del Grado en Matemáticas* dice:

> La memoria del TFG estará escrita en español, salvo un resumen inicial de al menos 500 palabras en inglés, cuya versión en español se incluirá a continuación.
>
> Para las memorias sujetas a evaluación por tribunal, a petición razonada del/la estudiante y con el visto bueno del tutor/a, la Comisión del TFG podrá autorizar que el resumen esté escrito en otro idioma, o que lo esté la memoria completa, en cuyo caso deberá incluirse un resumen de al menos 500 palabras en ese idioma con su versión en español.

Aquí se incluiría el resumen en inglés.

Resumen

Aquí se incluiría la traducción al español del resumen anterior.

CAPÍTULO 1

Introducción

En este capítulo se introducen los conceptos básicos necesarios para entender el posterior desarrollo del trabajo. En la sección 1.1 se realiza una introducción informal que abarca desde el Machine Learning hasta llegar al aprendizaje semi-supervisado. De manera complementaria, en la sección 1.2, se presenta la estructura del trabajo.

1.1. Conceptos iniciales

El **aprendizaje computacional**, o Machine Learning, tiene muchas definiciones, pero de manera intuitiva se puede entender como la rama de la ciencia encargada del diseño y construcción de algoritmos que, a partir de experiencia previa, normalmente dada como un conjunto de datos de entrenamiento, son capaces de aprender patrones que les permiten realizar predicciones o tomar decisiones sobre nuevos datos, es decir, les permite generalizar [22].

Esta experiencia previa viene generalmente dada en forma de ejemplos, que son instancias de los datos que usa el modelo o algoritmo, y que normalmente se representan como un vector $\mathbf{x}$ de características o atributos. Además, en algunos casos los ejemplos vienen acompañados de una etiqueta y, que no es más que un valor o categoría asociado a un vector $\mathbf{x}$. El conjunto de todos los ejemplos usados por un algoritmo para mejorar su rendimiento se conoce como el conjunto de entrenamiento [22].

Dentro del aprendizaje computacional, se pueden distinguir tres grandes categorías:

- Por un lado, tenemos el **Aprendizaje Supervisado**, aquellas aplicaciones en las que el conjunto de entrenamiento está formado por *datos etiquetados*, es decir, pares $(\mathbf{x}, y)$ de vectores de entrada con sus correspondientes etiquetas de salida [6], y el objetivo es obtener un modelo que sea capaz de predecir la etiqueta y de nuevos vectores $\mathbf{x}$ no vistos durante el entrenamiento. Dentro de esta categoría se presenta otra gran subdivisión entre los casos en los que la etiqueta a predecir está dentro de un conjunto finito de categorías, lo que se conoce como *problemas de clasificación*; y los casos en los que la etiqueta a predecir es una variable continua, lo que se conoce como *problemas de regresión* [6].
- Por otro lado, tenemos el **Aprendizaje No Supervisado** cuando el conjunto de datos de entrenamiento consiste en *datos no etiquetados* –ejemplos de vectores de entrada $\mathbf{x}$ sin sus correspondientes etiquetas de salida [6]. En este caso, el objetivo puede ser encontrar grupos de ejemplos similares en los datos, *clustering*; determinar la distribución de los datos en el espacio de entrada, *density estimation*; o proyectar los datos de un espacio de alta dimensión a uno más sencillo, *dimensionality reduction* [6].

- Además de estas dos categorías, existe una tercera gran categoría, el **Aprendizaje por Refuerzo**. Este, a diferencia de los anteriores donde se toma un conjunto de entrenamiento estático, se basa en la idea de que el algoritmo de aprendizaje interactúe activamente con el entorno, recibiendo recompensas inmediatas por cada acción tomada. El objetivo en este tipo de aprendizaje es maximizar la recompensa a lo largo de un curso de interacciones con el entorno [22].

Sin embargo, en este trabajo no nos vamos a centrar en una de estas grandes ramas del aprendizaje computacional, sino que vamos a tratar un tipo de aprendizaje que se encuentra a medio camino entre los tipos de aprendizaje estáticos vistos. El **Aprendizaje Semi-Supervisado**, la rama del aprendizaje computacional que usa tanto datos etiquetados como no etiquetados para realizar distintas tareas de aprendizaje [28].

Aun así, esta definición de aprendizaje semi-supervisado es demasiado general para los objetivos de este trabajo, es por esto que se va a tomar el enfoque propuesto en *A taxonomy for semi-supervised learning methods, Seeger, 2006* [25], donde se considera al **aprendizaje semi-supervisado** como una extensión del aprendizaje supervisado que se beneficia de la presencia de datos no etiquetados. De manera complementaria, se denomina **aprendizaje semi-no-supervisado** a las tareas de aprendizaje no supervisado que se benefician de la presencia de datos etiquetados, que quedan fuera del alcance de este trabajo.

El aprendizaje semi-supervisado es muy atractivo por dos razones: por un lado, porque puede potencialmente utilizar tanto datos etiquetados como no etiquetados para lograr un mejor rendimiento que el aprendizaje supervisado. Y, por otro lado, porque puede lograr el mismo nivel de rendimiento que el aprendizaje supervisado, pero con menos instancias etiquetadas [35]. Ambos motivos son muy interesantes debido al coste que supone etiquetar los datos –anotadores expertos en la materia en específico–, frente a la abundancia y el bajo coste de obtención de los datos no etiquetados.

Finalmente, destacar que dentro del aprendizaje semi-supervisado, al ser este una extensión del aprendizaje supervisado, se vuelve a encontrar la subdivisión entre problemas de clasificación y regresión. Este trabajo se va a centrar únicamente en el aprendizaje semi-supervisado para problemas de clasificación, aunque muchos de los resultados que se presentan también se pueden aplicar a problemas de regresión.

1.2. Estructura del trabajo

Una vez realizada esta introducción, finalizamos este capítulo presentando la estructura con la que se ha organizado el resto del trabajo:

- En el capítulo 2 se presenta formalmente el aprendizaje semi-supervisado, definiendo los conceptos básicos y presentando las diferentes suposiciones que se hacen sobre los datos para que el aprendizaje semi-supervisado pueda ser efectivo.
- En el capítulo 3 se exponen diferentes maneras de organizar los algoritmos de aprendizaje semi-supervisado, para concluir presentando una taxonomía de los mismos.
- En el capítulo 4 se procede entonces a realizar un estudio teórico de los métodos envolventes y sus diferentes variantes.

- En el capítulo 5 se presentan los métodos basados en preprocesamiento de los datos no etiquetados.
- En el capítulo 6 se presentan los métodos intrínsecamente semi-supervisados.
- Finalmente, en el capítulo 7 se presentan los métodos trasductivos.

CAPÍTULO 2 Introducción al Aprendizaje Semi-Supervisado

En este capítulo se da un enfoque formal a los conceptos introducidos en el capítulo anterior. En la sección 2.1 se presenta una definición formal de la clasificación semi-supervisada, el concepto sobre el que se articula el trabajo. Posteriormente, en la sección 2.3, se presentan las suposiciones que hay detrás del aprendizaje semi-supervisado, y que son necesarias para entender cómo este puede obtener mejor rendimiento que el aprendizaje supervisado, y cuándo es así.

2.1. Problema de clasificación semi-supervisado

Una vez introducida la idea en la que se basa el aprendizaje semi-supervisado, y dado que este trabajo se va a centrar en su aplicación a problemas de clasificación, es necesario definir de manera formal este problema. Para ello, vamos a hacer uso de una serie de definiciones que se encuentran en el libro de *Introduction to Semi-Supervised Learning, Xiaojin Zhu y Goldberg, 2009* [35], y que se presentan a continuación.

Como primer paso se presenta la definición del aprendizaje supervisado, que, como se ha mencionado, es la base del aprendizaje semi-supervisado:

Definición 2.1.1 (Aprendizaje Supervisado) Sea $\mathscr{X}$ el dominio de las instancias, e $\mathscr{Y}$ el dominio de las etiquetas. Sea $P(\mathbf{x}, y)$ una distribución de probabilidad conjunta (desconocida) sobre las instancias y etiquetas $\mathscr{X} \times \mathscr{Y}$. Dado un conjunto de entrenamiento $\{(\mathbf{x}_i, y_i)\}_{i=1}^{n}$ con $(\mathbf{x}_i, y_i) \overset{\text{i.i.d.}}{\sim} P(\mathbf{x}, y)$ (independiente e idénticamente distribuido), el aprendizaje supervisado entrena, usando este conjunto, una función $f : \mathscr{X} \to \mathscr{Y}$[1] en alguna familia de funciones $\mathscr{F}$, con el objetivo de que $f(\mathbf{x})$ prediga la verdadera etiqueta y en datos futuros $\mathbf{x}$, donde $(\mathbf{x}, y) \overset{\text{i.i.d.}}{\sim} P(\mathbf{x}, y)$ también.

A partir de esta definición de aprendizaje supervisado, se puede entonces definir el problema de clasificación:

Definición 2.1.2 (Clasificación) La clasificación es el problema de aprendizaje supervisado con clases discretas $\mathscr{Y}$. La función f se denomina clasificador.

[1] A lo largo de este trabajo, los términos modelo y algoritmo se utilizarán de manera intercambiable. No obstante, en sentido estricto, el modelo hace referencia a la función matemática (o hipótesis) que evalúa un vector de entrada para predecir una etiqueta, mientras que el algoritmo (de aprendizaje) es el procedimiento de optimización empleado para ajustar los parámetros de dicha función a partir del conjunto de entrenamiento.

Finalmente, podemos definir el problema de clasificación semi-supervisado como una extensión del supervisado de la siguiente manera:

Definición 2.1.3 (Clasificación Semi-Supervisada) Bajo las condiciones del aprendizaje supervisado, se expande el conjunto de entrenamiento para que consista tanto en l instancias etiquetadas $\{(\mathbf{x}_i, y_i)\}_{i=1}^{l}$, con $(\mathbf{x}_i, y_i) \overset{\text{i.i.d.}}{\sim} P(\mathbf{x}, y)$, como en u instancias no etiquetadas $\{\mathbf{x}_j\}_{j=l+1}^{l+u}$, con $(\mathbf{x}_j) \overset{\text{i.i.d.}}{\sim} P(\mathbf{x})$, siendo este término la distribución marginal de $P(\mathbf{x}, y)$ relativa al dominio de las instancias. El objetivo sigue siendo entrenar una función $f : \mathscr{X} \to \mathscr{Y}$ en alguna familia de funciones $\mathscr{F}$, con el objetivo de que $f(\mathbf{x})$ prediga la verdadera etiqueta y en datos futuros $\mathbf{x}$, donde $(\mathbf{x}, y) \overset{\text{i.i.d.}}{\sim} P(\mathbf{x}, y)$.

Observación 2.1.4 Se asume típicamente que hay muchos más datos no etiquetados que etiquetados, es decir, $u \gg l$.

Observación 2.1.5 Al tratarse del problema de clasificación, el dominio de las etiquetas $\mathscr{Y}$ es un conjunto discreto. Si fuera continuo, estaríamos hablando de un problema de regresión semi-supervisada que queda fuera del alcance de este trabajo.

Con estas definiciones, queda establecido el problema sobre el que se articula este trabajo. En la siguiente sección se introducen los conceptos de función de pérdida y regularización, herramientas fundamentales para formalizar el proceso de aprendizaje. Posteriormente, en la sección 2.3, se explicará por qué y bajo qué condiciones el aprendizaje semi-supervisado puede superar al aprendizaje supervisado.

2.2. Funciones de pérdida y regularización

Una vez definido el problema de clasificación semi-supervisada, es necesario introducir los conceptos fundamentales que permiten formalizar el proceso de aprendizaje. En particular, se necesita un mecanismo para medir la calidad de las predicciones de un clasificador y un criterio para seleccionar el mejor clasificador dentro de una familia dada. Las definiciones que se presentan a continuación se basan en las formulaciones de *Introduction to Semi-Supervised Learning, Zhu y Goldberg, 2009* [35] y *Foundations of Machine Learning, Mohri et al., 2018* [22].

Definición 2.2.6 (Función de pérdida) Sea $\mathscr{X}$ el dominio de las instancias, $\mathscr{Y}$ el dominio de las etiquetas y $f : \mathscr{X} \to \mathscr{Y}$ un clasificador. Una **función de pérdida** es una función $c : \mathscr{X} \times \mathscr{Y} \times \mathscr{Y} \to \mathbb{R}_{\geq 0}$ que mide el coste de predecir $f(\mathbf{x})$ cuando la etiqueta real es y. Se denota $c(\mathbf{x}, y, f(\mathbf{x}))$.

Ejemplo 2.2.7 En problemas de clasificación, una función de pérdida habitual es la **pérdida 0-1**, definida como $c(\mathbf{x}, y, f(\mathbf{x})) = \mathbf{1}_{f(\mathbf{x}) \neq y}$, que asigna coste 1 si la predicción es incorrecta y 0 en caso contrario.

Una vez establecida la noción de pérdida individual, se puede definir una medida global del rendimiento del clasificador:

Definición 2.2.8 (Riesgo) Dado un clasificador $f \in \mathscr{F}$ y una función de pérdida c, el **riesgo** o **error de generalización** de f se define como la pérdida esperada bajo la distribución conjunta $P(\mathbf{x}, y)$:

$$R(f) = \mathbb{E}_{(\mathbf{x},y)\sim P}\left[c(\mathbf{x}, y, f(\mathbf{x}))\right]$$

El clasificador óptimo es aquel que minimiza el riesgo: $f^* = \arg\min_{f\in\mathscr{F}} R(f)$ [35].

Observación 2.2.9 El riesgo $R(f)$ no se puede calcular directamente, puesto que la distribución $P(\mathbf{x}, y)$ es desconocida. Es por esto que se recurre a una aproximación basada en los datos disponibles.

Definición 2.2.10 (Riesgo empírico) Dado un conjunto de entrenamiento etiquetado $\{(\mathbf{x}_i, y_i)\}_{i=1}^{l}$, el **riesgo empírico** de un clasificador f se define como:

$$\hat{R}(f) = \frac{1}{l}\sum_{i=1}^{l} c(\mathbf{x}_i, y_i, f(\mathbf{x}_i))$$

Observación 2.2.11 El riesgo empírico es un estimador insesgado del riesgo real, es decir, $\mathbb{E}[\hat{R}(f)] = R(f)$ [22]. Esto justifica su uso como aproximación del riesgo verdadero.

El principio de **minimización del riesgo empírico** (ERM, por sus siglas en inglés) propone seleccionar el clasificador que minimice esta cantidad:

$$f_{\text{ERM}} = \arg\min_{f\in\mathscr{F}} \hat{R}(f) \tag{2.2.1}$$

Sin embargo, este principio tiende a producir modelos que se *sobreajustan* a los datos de entrenamiento, es decir, que memorizan las particularidades del conjunto de entrenamiento perdiendo capacidad de generalización a datos nuevos [22]. Para mitigar este problema, se introduce un término adicional que penaliza la complejidad del clasificador, dando lugar al marco de **minimización del riesgo regularizado**:

Definición 2.2.12 (Minimización del riesgo regularizado) Dada una función de pérdida c, un funcional de regularización $\Omega : \mathscr{F} \to \mathbb{R}_{\geq 0}$ que mide la complejidad del modelo, y un parámetro de regularización $\lambda > 0$, el problema de minimización del riesgo regularizado consiste en encontrar:

$$f^* = \arg\min_{f\in\mathscr{F}} \left[\hat{R}(f) + \lambda\,\Omega(f)\right] = \arg\min_{f\in\mathscr{F}} \left[\frac{1}{l}\sum_{i=1}^{l} c(\mathbf{x}_i, y_i, f(\mathbf{x}_i)) + \lambda\,\Omega(f)\right] \tag{2.2.2}$$

Este marco es de gran utilidad tanto para comprender las suposiciones del aprendizaje semi-supervisado como para formalizar los métodos que se presentarán en capítulos posteriores. En particular, en el contexto semi-supervisado, el funcional de regularización puede incorporar información de los datos no etiquetados, adoptando la forma $\Omega(f) = \Omega_{\text{sup}}(f) + \lambda'\,\Omega_{\text{ssl}}(f)$, donde Ω_{sup} es un regularizador estándar y Ω_{ssl} es un término que codifica información sobre la estructura de $P(\mathbf{x})$, estimada a partir de los datos no etiquetados [35].

2.3. Suposiciones del aprendizaje semi-supervisado

Aunque no todos los métodos de aprendizaje semi-supervisado se basan explícitamente en probabilidades, resulta útil para entender cómo el aprendizaje semi-supervisado puede obtener mejor rendimiento que el aprendizaje supervisado, asumir que los métodos representan las hipótesis como $p(y \mid \mathbf{x})$ y los datos no etiquetados como $p(\mathbf{x})$. La clave para entender el papel de los datos no etiquetados reside en la descomposición de la distribución conjunta:

$$p(\mathbf{x}, y) = p(y \mid \mathbf{x})\, p(\mathbf{x})$$

En el aprendizaje supervisado, el objetivo es estimar $p(y \mid \mathbf{x})$, y la distribución marginal $p(\mathbf{x})$ no aporta información adicional directa sobre $p(y \mid \mathbf{x})$. Sin embargo, en el aprendizaje semi-supervisado, la información que proporcionan los datos no etiquetados sobre $p(\mathbf{x})$ se puede aprovechar de dos posibles formas: o bien a través de modelos generativos que estiman la distribución conjunta $p(\mathbf{x}, y)$, la cual comparte parámetros con $p(\mathbf{x})$; o bien modificando la función objetivo (por ejemplo, el funcional de regularización Ω_{ssl} de la Sección 2.2) para incorporar términos que dependan de $p(\mathbf{x})$ [36].

No obstante, para que el conocimiento sobre $p(\mathbf{x})$ sea realmente útil para inferir $p(y \mid \mathbf{x})$, debe existir una relación estructural entre ambas distribuciones. Es por esto que detrás de cada método de aprendizaje semi-supervisado hay una serie de suposiciones sobre cómo estas distribuciones se relacionan entre sí. Pero, antes de exponer estas suposiciones, conviene clarificar algunos conceptos fundamentales sobre la geometría de los datos:

- **Densidad:** Se refiere a la concentración de puntos en una región determinada del dominio de las instancias $\mathscr{X}$. Formalmente, las regiones de alta densidad son aquellas donde el valor de la función de densidad de probabilidad $p(\mathbf{x})$ es elevado, mientras que las de baja densidad corresponden a valores reducidos de $p(\mathbf{x})$.
- **Clúster:** De manera intuitiva, un clúster es un conjunto de puntos que, bajo una cierta métrica, se encuentran significativamente más cerca entre sí que del resto de los datos [6]. De manera formal, se puede entender como cada uno de los subconjuntos C_k, para $\mathrm{k} \in \{1, 2, \ldots, K\}$, de una partición $\{C_1, C_2, \ldots, C_K, C_{\mathrm{K+1}}\}$ del dominio $\mathscr{X}$ de tal manera que los puntos de cada subconjunto, con excepción del último, cumplen alguna propiedad métrica [18]. Sin embargo, la propiedad concreta depende de cada algoritmo de clustering es específico [11].
- **Frontera de decisión:** Es la superficie o hipersuperficie en el espacio de entrada $\mathscr{X}$ que separa las regiones asignadas a diferentes clases por un clasificador f [6].
- **Variedad (*manifold*):** Un espacio topológico $\mathscr{M}$ que es localmente homeomorfo a $\mathbb{R}^d$, es decir, para cada punto $p \in \mathscr{M}$ existe un entorno abierto U tal que $p \in U$ y un homeomorfismo $\varphi : U \to V \subseteq \mathbb{R}^d$. En el contexto de este trabajo, se considera que los datos residen en (o cerca de) una variedad $\mathscr{M}$ de dimensión d sumergida en el espacio de entrada $\mathscr{X} \subseteq \mathbb{R}^D$ con $d \ll D$ [8].

Una vez presentados estos conceptos, se pueden entonces exponer cuáles son las suposiciones que permiten que los datos no etiquetados aporten información útil. Estas suposiciones se presentan tal y como aparecen en *Semi-supervised learning, Chapelle et al., 2006* [8]:

- **Suposición de suavidad (semi-supervisada)**: Si dos puntos $\mathbf{x}_1$ y $\mathbf{x}_2$ se encuentran en una región de alta densidad de $p(\mathbf{x})$ y están cercanos entre sí, entonces sus correspondientes salidas y_1 e y_2 también deben estarlo. Nótese la diferencia con la suposición de suavidad clásica del aprendizaje supervisado, que simplemente requiere que puntos cercanos tengan salidas cercanas: aquí la suavidad está modulada por la densidad, lo que significa que la función de predicción puede cambiar abruptamente en regiones de baja densidad.
- **Suposición de clústers**: Si dos puntos $\mathbf{x}_1$ y $\mathbf{x}_2$ pertenecen a un mismo clúster de $p(\mathbf{x})$, es probable que compartan la misma etiqueta, es decir, $p(y \mid \mathbf{x}_1) \approx p(y \mid \mathbf{x}_2)$. Bajo esta suposición, los datos no etiquetados, al revelar la estructura de clústers de $p(\mathbf{x})$, proporcionan información sobre las fronteras entre clases.
- **Suposición de baja densidad**: La frontera de decisión del clasificador debe pasar por regiones de baja densidad de $p(\mathbf{x})$. Esta suposición es, de hecho, una reformulación equivalente de la suposición de clústers: si los puntos de un mismo clúster comparten etiqueta, entonces las fronteras entre clases necesariamente se hallan en las regiones de baja densidad que separan los clústers.
- **Suposición de variedad subyacente**: Los datos se encuentran (aproximadamente) en una variedad $\mathscr{M}$ de dimensión $d \ll D$ dentro de la representación de alta dimensión del espacio de entrada $\mathscr{X} \subseteq \mathbb{R}^D$. Esta suposición permite reducir la *maldición de la dimensionalidad*: en lugar de trabajar en el espacio ambiente $\mathbb{R}^D$, las distancias y relaciones entre puntos se pueden calcular sobre la variedad $\mathscr{M}$, donde la dimensión intrínseca d es mucho menor [8]. Los datos no etiquetados, al ser abundantes, permiten estimar mejor la geometría de $\mathscr{M}$.

Observación 2.3.13 Las suposiciones de clústers y de baja densidad, aunque equivalentes desde un punto de vista teórico, sugieren diferentes estrategias algorítmicas. La suposición de clústers lleva naturalmente a métodos que primero identifican la estructura de los datos (como los basados en grafos), mientras que la suposición de baja densidad sugiere métodos que buscan directamente fronteras de decisión en regiones de baja densidad (como los métodos de máximo margen).

Destacar que si la suposición en la que se basa un método de aprendizaje semi-supervisado no se cumple, el uso de datos no etiquetados puede ser contraproducente y empeorar el rendimiento respecto al aprendizaje supervisado [35]. Este fenómeno, conocido informalmente como *degradación del rendimiento*, subraya la importancia de verificar o al menos controlar la validez de las suposiciones a la hora de aplicar un método semi-supervisado a un problema concreto.

CAPÍTULO 3 Taxonomía de métodos de aprendizaje semi-supervisado

En este capítulo se va a presentar una taxonomía de los diferentes métodos de clasificación bajo el enfoque de aprendizaje semi-supervisado. Su objetivo es permitir un entendimiento claro de la base de los diferentes métodos con el fin de facilitar su comprensión y estudio. Para realizarla se ha utilizado como estructura principal la taxonomía presentada en el artículo *A survey on Semi-supervised Learning, van Engelen y Hoos, 2020* [28] a la que se le han realizado ciertas modificaciones debidamente motivadas buscando una mejor comprensión y adaptación a los objetivos de este trabajo.

3.1. Características discriminantes

Antes de exponer la taxonomía, se van a introducir posibles características discriminantes que pueden ser utilizadas para clasificar los diferentes métodos de aprendizaje semi-supervisado, explicando para cada una de ellas su significado y su importancia a la hora de clasificar los diferentes métodos. Finalmente, la taxonomía se creará a partir de estas características, asignando una jerarquía de niveles a las mismas y agrupando los diferentes métodos en las hojas del árbol resultante.

3.1.1. Suposición sobre la relación entre los datos etiquetados y no etiquetados

Como se explicó al final del último capítulo las diferentes suposiciones sobre cómo se relacionan los datos etiquetados y no etiquetados es una característica básica a la hora de entender los diferentes métodos de aprendizaje semi-supervisado. Es por esto que surge rápidamente como una posible característica discriminante a la hora de clasificar los diferentes métodos, como por ejemplo se hace en *Semi-supervised learning, Chapelle et al., 2006* [8].

Sin embargo, clasificar en base a esta característica no es ni tan sencillo ni tan claro como parece a primera vista. Por un lado, las suposiciones teóricas a menudo se solapan conceptualmente, o son compartidas por diferentes algoritmos cuya implementación matemática difiere radicalmente. Además, hay ciertos algoritmos de aprendizaje semi-supervisado – como los métodos *wrapper,* que se presentarán más adelante–, que no se basan en suposiciones explícitas sobre los datos, por lo que una clasificación por esta característica resulta incompleta. Por tanto, aunque es importante conocer la suposición de cada algoritmo a la hora de trabajar con él, una clasificación en base a esta característica puede no resultar suficientemente esclarecedora para entender las ideas y fundamentos en los que se basa.

3.1.2. Inductivo vs transductivo

Una segunda característica discriminante, que es, de hecho, un estándar de facto en la literatura a la hora de clasificar los diferentes métodos de aprendizaje semi-supervisado, es la distinción entre métodos inductivos y transductivos. Esta distinción se basa en el objetivo final del método: Por un lado, los **métodos inductivos** buscan aprender una función de predicción $f : \mathscr{X} \to \mathscr{Y}$ que pueda ser aplicada a cualquier punto del espacio de entrada $\mathscr{X}$, mientras que los **métodos transductivos** se centran en realizar predicciones únicamente para un conjunto específico de puntos de *prueba*, los puntos no etiquetados de la entrada [8].

Esta distinción es crucial pues refleja el objetivo final de un método, y por tanto influye en su diseño y en las técnicas utilizadas para su implementación. Mientras que los métodos transductivos pueden aprovechar al máximo la información de los datos no etiquetados para mejorar sus predicciones, los métodos inductivos deben ser capaces de generalizar a nuevos puntos de entrada, lo que puede requerir técnicas adicionales para evitar el sobreajuste a los datos de entrenamiento.

3.1.3. Generativo vs discriminante

Otro criterio de clasificación estándar en la literatura, sobre todo la más clásica, es la distinción entre métodos generativos y discriminantes. Esta distinción se presentó de manera implícita en el capítulo anterior, cuando se explicó cómo la relación entre $p(x)$ y $p(y|x)$ es la razón por la que el aprendizaje semi-supervisado mejora sus predicciones.

Este criterio clasifica los métodos en dos tipos: Los métodos **generativos** que buscan aprender un modelo de la probabilidad conjunta de los datos $p(x,y)$, de las entradas x, y de las etiquetas y, para hacer predicciones usando la regla de Bayes para calcular $p(y|x)$ y seleccionando la etiqueta más probable. Y los métodos **discriminantes** que se centran en modelar directamente la probabilidad a posteriori $p(y|x)$ o aprenden un mapeo directo de las entradas a las etiquetas [23].

Esta distinción es muy útil pues permite comprender los fundamentos que hay detrás de un método, antes de abordar los detalles de su implementación concreta. Sin embargo, no es una distinción perfecta, ya que no contempla todos los posibles métodos, en particular, los métodos que se basan en la geometría o topología de los datos –como los métodos basados en grafos o variedades–.

3.1.4. Basado en el uso de los datos no etiquetados

Finalmente, se presenta una última característica discriminante, la forma en la que los diferentes algoritmos hacen uso de los datos no etiquetados. Esta característica es la base de la taxonomía que se presenta en el trabajo de van Engelen y Hoos [28], y por ende es la base de la taxonomía presentada en este trabajo.

Esta característica clasifica los algoritmos de aprendizaje semi-supervisado en tres grupos:

- Los métodos **Envolventes** o **Wrapper**, métodos que se basan en un paso de pseudo-etiquetado. Es decir, se basan en un proceso que utiliza uno o varios algoritmos supervisados, que parten solo con los datos etiquetados, y va iterativamente etiquetando parte de

los datos no etiquetados usando estos algoritmos, para posteriormente reentrenarlos usando estas nuevas etiquetas, sin que estos distingan la diferencia entre etiquetas iniciales y etiquetadas por el proceso.

- Los métodos basados en **preprocesado no supervisado**, aquellos que se basan en un pipeline de dos etapas: una primera etapa de preprocesamiento no supervisado, que puede ser por ejemplo una etapa de reducción de dimensionalidad o de extracción de características, seguida de una etapa de clasificación donde un algoritmo supervisado, agnóstico a la etapa anterior, se entrena con los datos etiquetados y la representación obtenida en la primera etapa.
- Los métodos **intrínsecamente semi-supervisados**, que directamente optimizan una función objetivo haciendo uso tanto de los datos etiquetados como los no etiquetados. Por tanto, no necesitan pasos intermedios ni algoritmos supervisados de base, sino que usualmente son extensiones de estos algoritmos supervisados que integran de manera explícita los datos no etiquetados en su formulación matemática.

El empleo de esta característica para clasificar es de gran utilidad, ya que permite realizar una división funcional de los métodos clara y sencilla, que permite entender la estructura general detrás de un método, sin necesidad de abordar los fundamentos matemáticos del mismo.

3.2. Taxonomía

Basándose en todo lo explicado anteriormente, se presenta en la figura 3.1 una taxonomía de, principalmente, dos niveles de profundidad, que profundiza a un tercer nivel en los métodos inductivos intrínsecamente semi-supervisados.

1. Primero, se distingue entre los métodos inductivos o transductivos.
2. Una vez clasificados en base a esta característica, se considera cómo el algoritmo hace uso de los datos no etiquetados.
3. Finalmente, dentro de los métodos inductivos e intrínsecamente semi-supervisados, se realiza una última distinción basada en el fundamento matemático del método, distinguiendo entre métodos generativos y discriminantes, añadiendo además una tercera rama para los métodos geométricos (basados en variedades).

Cabe notar, como se viene comentando, la similitud entre esta taxonomía y la presentada en el trabajo de van Engelen y Hoos [28]. Se va primero a explicar las similitudes, que son las que forman la base de la taxonomía presentada, para posteriormente explicar las diferencias que hay entre ambas.

La primera similitud es la división inicial de la taxonomía en métodos inductivos y transductivos, esta división, estándar de facto en la literatura, es, como se ha comentado en la sección 3.1.2, crucial para entender el objetivo final de un método y por tanto es lo primero que se ha de tener en cuenta a la hora de abordar un método de aprendizaje semi-supervisado.

La segunda similitud, y la base de la taxonomía de van Engelen y Hoos [28], es la división de los métodos inductivos en base a cómo hacen uso de los datos no etiquetados. Esta división, como se ha comentado en la sección 3.1.4, permite entender la estructura general detrás de un método, siendo una herramienta muy útil para comprender de antemano un método o algoritmo.

Estas similitudes fundamentan la base de una taxonomía sólida y clara, que permite entender como va actuar de manera general un método, sin necesidad de abordar los detalles matemáticos del mismo.

Sin embargo, y aquí aparece la primera diferencia, para realizar esta taxonomía, se ha decidido seguir otro estándar de la literatura, y distinguir entre los métodos discriminantes y generativos, como se ha explicado en la sección 3.1.3. Esta distinción permite entender mejor los fundamentos matemáticos sobre los que se basa cada método, y aunque no engloba todos los métodos, al realizarse únicamente dentro de los métodos inductivos intrínsecamente semi-supervisados, no resulta tan limitante como si se realizara a un nivel más alto de la taxonomía.

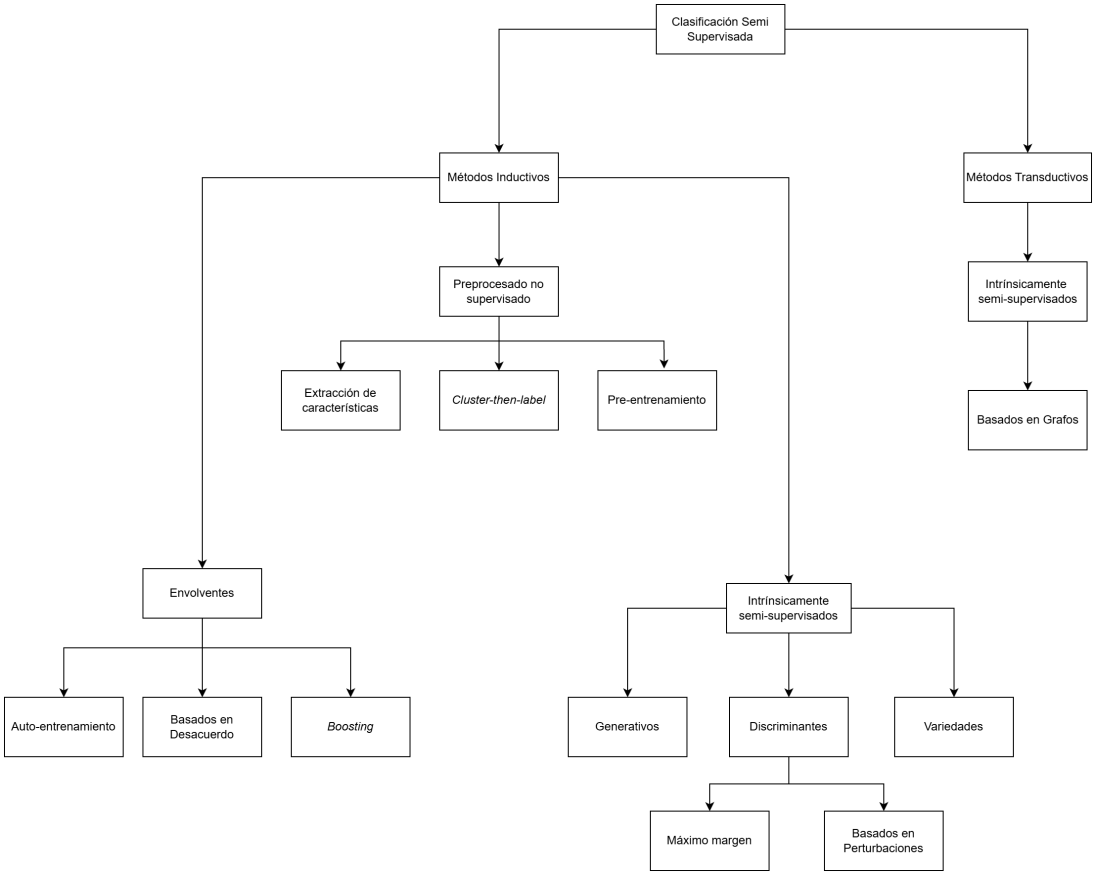

Figura 3.1: Taxonomía de métodos de aprendizaje semi-supervisado.

De esta forma, la taxonomía propuesta reorganiza los métodos intrínsecos en tres ramas diferenciadas: la familia de métodos generativos; la de métodos discriminantes —que agrupa bajo un mismo paraguas conceptual a los enfoques de máximo margen y a los basados en perturbaciones—; y finalmente, una categoría independiente para los métodos basados en variedades, reconociendo así su naturaleza geométrica distintiva.

Además de esta diferencia, se han añadido dos cambios menores:

Por un lado, para mantener la simetría entre los métodos inductivos y transductivos, se ha añadido en la rama de los métodos transductivos la división sobre el uso de los datos no etiquetados, aunque en este caso solo hay algoritmos intrínsecamente semi-supervisados. De esta manera se mantiene la simetría entre ambas ramas, y permite tener una mayor comprensión con tan solo ver la figura.

Por último, dentro de los métodos envolventes, se ha cambiado el nombre de la familia presentada por van Engelen y Hoos [28] como co-training por *métodos basados en desacuerdo*. De esta manera, se evita nombrar a la familia por un algoritmo en concreto dentro de esta, el algoritmo de co-training de Blum y Mitchell [7]. El objetivo del nuevo nombre, que se desarrollará en profundidad en el siguiente capítulo, es reflejar la idea general de esta familia de métodos tal y como se presenta en el trabajo de Zhou y Li [34].

CAPÍTULO 4

Métodos envolventes

La familia de métodos envolventes es de las más viejas y ampliamente conocidas dentro del aprendizaje semi-supervisado [34]. Como se ha explicado a la hora de presentar esta categoría en la taxonomía (Sección 3.1.4), estos métodos se basan en la idea del *pseudoetiquetado*, un procedimiento iterativo donde cada iteración consiste en dos pasos: primero se entrena a uno o varios clasificadores supervisados con los datos etiquetados originales y posiblemente con datos etiquetados de iteraciones anteriores, y luego se utilizan estos clasificadores para predecir etiquetas para los datos no etiquetados, las predicciones de mayor confianza son entonces añadidas al conjunto de datos etiquetados, convirtiéndose en datos pseudoetiquetados. Este proceso se repite hasta que se cumpla un criterio de parada, como por ejemplo un número máximo de iteraciones o una mejora mínima en el rendimiento del modelo.

Esta familia de métodos tiene varias peculiaridades que vamos a exponer a continuación:

- En primer lugar, tienen una propiedad clave que es la de poder ser aplicados a cualquier, o casi cualquier, modelo supervisado subyacente. Esto les otorga de una gran flexibilidad pues introducen el uso de los datos no etiquetados en el proceso de entrenamiento de manera sencilla y directa.

- Por otro lado, cabe recalcar que no se basan por sí mismos en ningún supuesto específico sobre la distribución de los datos o las relaciones entre los datos etiquetados y no etiquetados, sino que dependen de los supuestos que tengan los modelos supervisados subyacentes. Esto les otorga gran versatilidad, y los convierte en candidatos idóneos para mejorar el rendimiento en problemas en los que algún algoritmo supervisado ya trabaja bien.

- Sin embargo tienen una suposición importante: los métodos envolventes asumen que las predicciones, al menos la de mayor confianza, son correctas. En el caso de que esto no se cumpla, se puede producir un efecto de arrastre donde los errores se van acumulando a lo largo de las iteraciones, lo que puede resultar en una degradación del rendimiento del modelo [35].

Se van a presentar a continuación tres métodos o familias de métodos representativas de los métodos envolventes: el auto-entrenamiento, los métodos basados en desacuerdo y los métodos basados en boosting.

4.1. Auto-entrenamiento

El auto-entrenamiento es la versión más simple de los métodos envolventes y una de las más antiguas [32]. En este método se toma la idea del pseudoetiquetado y se aplica a un único clasificador supervisado, que se entrena con los datos etiquetados originales y luego se utiliza para predecir etiquetas para los datos no etiquetados, usando las predicciones de mayor confianza reforzar el entrenamiento en la siguiente iteración.

A pesar de su simplicidad, se pueden realizar varias decisiones de diseño a la hora de implementar un método de auto-entrenamiento: el criterio para seleccionar las predicciones de mayor confianza, el número de predicciones a añadir en cada iteración o el criterio de parada. Estas decisiones pueden tener un impacto significativo en el rendimiento del método y pueden ser adaptadas a las características específicas del problema y los datos [28].

Se expone a continuación un algoritmo de auto-entrenamiento donde se pueden observar estas decisiones de diseño:

Algoritmo 4.1.1 Auto-entrenamiento
Entrada: Conjunto de datos etiquetados $L = \{(\mathbf{x}_i, y_i)\}_{i=1}^{l}$, conjunto de datos no etiquetados $U = \{\mathbf{x}_j\}_{j=l+1}^{l+u}$, clasificador supervisado f, criterio de selección de predicciones de mayor confianza C, criterio de parada S.
Salida: Clasificador entrenado f.
Procedimiento:
Mientras no se cumpla el criterio de parada S:

1. Entrenar el clasificador supervisado f con los datos etiquetados L.
2. Predecir etiquetas para los datos no etiquetados U utilizando el clasificador supervisado f, obteniendo un conjunto de predicciones P.
3. Seleccionar un subconjunto de predicciones P_{conf} de P utilizando el criterio de selección de predicciones de mayor confianza C.
4. Añadir las predicciones seleccionadas P_{conf} al conjunto de datos etiquetados L, convirtiéndolas en datos pseudoetiquetados.
5. Eliminar las predicciones seleccionadas P_{conf} del conjunto de datos no etiquetados U.

4.2. Métodos basados en desacuerdo

Los métodos basados en desacuerdo son la extensión natural del auto-entrenamiento a múltiples clasificadores supervisados. Su nombre proviene del hecho de que en cada iteración del pseudoetiquetado, una vez entrenados los clasificadores supervisados, estos no necesariamente están de acuerdo en sus predicciones para los datos no etiquetados, y se pueden aprovechar estas diferencias para seleccionar las predicciones que se van a añadir al conjunto de datos etiquetados. Por ejemplo, se pueden seleccionar las predicciones donde los clasificadores están de acuerdo, o por el contrario, predicciones donde algún clasificador tiene mucha confianza en su predicción, dependiendo de la estrategia que se quiera seguir [34].

Los métodos basados en desacuerdo utilizan los datos no etiquetados como una especie de puente para intercambiar información entre los modelos a entrenar. El origen de esta discrepancia en la información y la manera en la que esta es propagada divide estos métodos en varias familias que se presentan a continuación.

4.2.1. Múltiples vistas

Una de las primeras ideas que dieron forma a este tipo de métodos fue la de descomponer el dominio de las instancias $\mathscr{X}$ en lo que se conoce como vistas, es decir, subconjuntos disjuntos de las características, y entrenar un clasificador supervisado para cada vista, de esta manera cada clasificador tiene una información diferente sobre el problema y se pueden aprovechar estas diferencias para mejorar el rendimiento del modelo [34].

El primer método de este tipo fue el co-training presentado en *Combining labeled and unlabeled data with co-training, Blum and Mitchell, 1998* [7] donde se divide el dominio en dos vistas $\mathscr{X} = \mathscr{X}_1 \cup \mathscr{X}_2$ de tal manera que cada vector de características $\mathbf{x}$ se puede representar como $\mathbf{x} = (\mathbf{x}_1, \mathbf{x}_2)$ con $\mathbf{x}_1 \in \mathscr{X}_1$ y $\mathbf{x}_2 \in \mathscr{X}_2$. El algoritmo de co-training consiste en entrenar dos clasificadores supervisados f_1 y f_2 con las vistas $\mathscr{X}_1$ y $\mathscr{X}_2$ respectivamente, y luego utilizar cada clasificador para predecir etiquetas para los datos no etiquetados, añadiendo las predicciones de mayor confianza de cada clasificador al conjunto de datos etiquetados del otro clasificador. Este proceso se repite iterativamente hasta que se cumpla un criterio de parada.

Blum y Mitchell hacen dos suposiciones clave para que co-training funcione correctamente: la primera es que las vistas son condicionalmente independientes dado la etiqueta, es decir, $P(\mathbf{x}_1, \mathbf{x}_2 \mid y) = P(\mathbf{x}_1 \mid y)P(\mathbf{x}_2 \mid y)$, lo que implica que cada vista proporciona información complementaria sobre la etiqueta. La segunda suposición es que cada vista es suficiente para predecir la etiqueta, es decir, $P(y \mid \mathbf{x}_1) = P(y \mid \mathbf{x}_2) = P(y \mid \mathbf{x})$, lo que implica que cada vista por sí sola es capaz de predecir la etiqueta con un rendimiento razonable [35].

Sin embargo, se ha demostrado que los métodos de múltiples vistas pueden funcionar con suposiciones más débiles en la independencia [2]. Por ejemplo, la suposición de expansión [3] relaja la independencia condicional, exigiendo simplemente que los ejemplos no estén excesivamente correlacionados y que el clasificador no cometa errores con alta confianza. Desde un punto de vista de teoría de aprendizaje, esto garantiza que el error de generalización se reduce al forzar el acuerdo entre las vistas.

4.2.2. Una única vista

Sin embargo, hay una suposición aún mayor en co-training y los métodos basados en múltiples vistas, y es que en muchos casos del mundo real no existe una división natural del dominio en vistas. Es aquí donde surgen los métodos de una única vista, estos buscan generar múltiples clasificadores supervisados a partir de un dominio que en su origen no tiene una división en vistas.

Una primera idea puede ser la de generar las vistas artificialmente, y usar estas vistas para entrenar los diferentes clasificadores supervisados [30]. Otras formas más naturales de entrenar múltiples clasificadores supervisados con una única vista, pero introduciendo alguna fuente de aleatoriedad para que cada clasificador tenga una información diferente sobre el problema,

pueden ser utilizar diferentes parámetros para un mismo modelo [31] o utilizar directamente diferentes modelos supervisados [13].

4.2.3. Co-regularización

Una última idea, dentro de los métodos envolventes basados en desacuerdo, pero algo alejada del concepto del pseudoetiquetado, es la de **co-regularización**. Al igual que en los métodos anteriores se entrenan diferentes clasificadores, sin embargo aquí el pseudoetiquetado no es un proceso iterativo, sino que se introduce un término de regularización en la función de pérdida de cada clasificador que penaliza el desacuerdo entre los clasificadores en sus predicciones para los datos no etiquetados, de esta manera se fuerza a los clasificadores a estar de acuerdo en sus predicciones para los datos no etiquetados, lo que puede mejorar el rendimiento del modelo [26].

Veamos qué quiere decir esto. Haciendo uso del marco de minimización del riesgo regularizado introducido en la Sección 2.2, recordemos que el objetivo del aprendizaje supervisado regularizado es encontrar un clasificador f que minimice:

$$\hat{R}(f) + \lambda\,\Omega(f) = \frac{1}{l}\sum_{i=1}^{l} c(\mathbf{x}_i, y_i, f(\mathbf{x}_i)) + \lambda\,\Omega(f)$$

La co-regularización extiende este marco a múltiples clasificadores. Dados k clasificadores $f_1, f_2, \ldots, f_k$, se busca minimizar una función objetivo que combina el riesgo regularizado de cada clasificador con un término que penaliza el desacuerdo entre ellos en los datos no etiquetados. Formalmente, se busca el conjunto de clasificadores que minimice la siguiente expresión [26]:

$$\sum_{r=1}^{k}\left(\sum_{i=1}^{l} c(\mathbf{x}_i, y_i, f_r(\mathbf{x}_i)) + \lambda\,\Omega(f_r)\right) + \gamma \sum_{\substack{r,s=1\\ r\neq s}}^{k} \sum_{j=l+1}^{l+u} c(\mathbf{x}_j, f_r(\mathbf{x}_j), f_s(\mathbf{x}_j)) \tag{4.2.1}$$

Los dos primeros términos corresponden a la minimización del riesgo regularizado estándar de cada clasificador sobre los datos etiquetados. El tercer término es el aporte propio de la co-regularización: penaliza las discrepancias entre las predicciones de los distintos clasificadores sobre los datos no etiquetados, ponderadas por el hiperparámetro $\gamma > 0$. De esta manera, los datos no etiquetados actúan como un puente que fuerza el acuerdo entre los clasificadores, transfiriendo información entre ellos de forma implícita.

Destacar que aunque la co-regularización no se basa en un proceso iterativo de pseudoetiquetado, y aunque haya aparecido una función de pérdida que se minimiza, sigue siendo un método envolvente basado en desacuerdo, pues se siguen entrenando diferentes clasificadores supervisados con la información de los datos no etiquetados, y se sigue buscando aprovechar las diferencias entre los clasificadores para mejorar el rendimiento del modelo.

4.3. Boosting

Una última familia de métodos envolventes son aquellos que se basan en la idea del boosting. El boosting es una técnica de aprendizaje supervisado que se encuentra en el marco de

los métodos de ensamblado, los cuales se basan en la idea de entrenar múltiples modelos supervisados y luego combinar sus predicciones para obtener una predicción final. El boosting se diferencia de otros métodos de ensamblado en que los modelos supervisados se entrenan de manera secuencial, cada modelo se entrena para corregir los errores del modelo anterior, lo que puede mejorar el rendimiento del modelo [33].

En el contexto del aprendizaje semi-supervisado se han desarrollado varios métodos basados en el boosting, pues permiten introducir una etapa de pseudoetiquetado tras cada entrenamiento. El precursor de estos modelos fue SSMBoost [15], sin embargo, este método se basa en usar un algoritmo semi-supervisado de base, no en introducir el pseudoetiquetado tras cada entrenamiento, por lo que al no actuar de caja negra no es realmente un método envolvente. Posteriormente, se han desarrollado otros métodos basados en el boosting que sí introducen el pseudoetiquetado tras cada entrenamiento, como por ejemplo Assemble [5] o SemiBoost [21], ambos se diferencian en la forma de introducir el pseudoetiquetado en la siguiente iteración. Assemble toma un enfoque más directo pues hace una selección aleatoria con remplazamiento (*bootstraping*) sobre el conjunto formado por los datos etiquetados y pseudoetiquetados para generar el conjunto de datos etiquetados para la siguiente iteración. Por otro lado en SemiBoost se seleccionan las predicciones de mayor confianza para generar el conjunto de datos etiquetados para la siguiente iteración, la confianza de cada predicción individual se calcula usando un grafo local construido mediante similitud entre los datos de entrada.

CAPÍTULO 5 Preprocesado no-supervisado de datos

La siguiente gran familia que vamos a presentar es la de los métodos de preprocesado no-supervisado de datos. La idea de esta familia es usar los datos no etiquetados en un paso previo al entrenamiento del modelo supervisado, con el objetivo de mejorar el rendimiento del modelo o reducir su complejidad [28].

Esta idea contrasta tanto con los métodos envolventes, que introducen los datos no etiquetados en el entrenamiento mediante el pseudoetiquetado, como con los métodos de intrínsecos, que introducen los datos no etiquetados en el entrenamiento mediante la adición de términos no etiquetados a la función de pérdida. Esta diferencia es importante, ya que permite a los métodos de preprocesado no-supervisado de datos ser compatibles con cualquier modelo supervisado, sin necesidad de modificar su función de pérdida o su proceso de entrenamiento.

Cada método de esta familia se distingue del resto por como implementa este *paso previo*, aun así podemos identificar tres grandes categorías o subfamilias: Las basadas en la extracción de características, las basadas en el clustering y las basadas en el pre-entrenamiento de modelos.

5.1. Extracción de características

Los métodos de extracción de características buscan transformar los datos de entrada de su representación original a una representación diferente que mejore el rendimiento del modelo o reduzca la complejidad del entrenamiento. Esta transformación se puede realizar de diferentes maneras, como por ejemplo reduciendo la dimensionalidad de los datos, seleccionando características o generando nuevas características a partir de las originales.

Uno de los métodos más conocidos de esta categoría es el **Análisis de Componentes Principales** (PCA), que busca encontrar una nueva representación de los datos en un espacio de menor dimensionalidad, donde las nuevas dimensiones son combinaciones lineales de las dimensiones originales que capturan la mayor parte de la varianza de los datos [1]. El porqué de usar la varianza como criterio viene del siguiente teorema [27]:

Teorema 5.1.1 *Sea $\mathscr{X} \subseteq \mathbb{R}^d$ el dominio de las instancias y $\mathscr{D} = \{\mathbf{x}_1, \mathbf{x}_2, \ldots, \mathbf{x}_n\}$ un conjunto de datos. Entonces, la proyección de los datos sobre un subespacio de dimensión $d' < d$ que maximiza la varianza de los datos proyectados es la misma proyección que minimiza la distancia entre los datos originales y su aproximación en el nuevo subespacio, es decir, la que minimiza la siguiente expresión:*

$$E = \frac{1}{n} \sum_{i=1}^{n} \|\mathbf{x}_i - \tilde{\mathbf{x}}_i\|^2$$

donde $\tilde{\mathbf{x}}_i$ es la proyección ortogonal de $\mathbf{x}_i$ sobre el subespacio de dimensión d'.

Demostración.
Sin perder generalidad, podemos asumir que los datos están centrados en el origen, es decir, que $\frac{1}{n}\sum_{i=1}^{n}\mathbf{x}_i = \mathbf{0}$.

Sea $\{\mathbf{v}_j\}_{j=1}^{d'}$ una base ortonormal arbitraria que genera este nuevo subespacio. La proyección ortogonal de una muestra original $\mathbf{x}_i$ sobre dicho subespacio viene dada por la siguiente combinación lineal:

$$\tilde{\mathbf{x}}_i = \sum_{j=1}^{d'}(\mathbf{x}_i \cdot \mathbf{v}_j)\mathbf{v}_j$$

Cualquier punto original puede descomponerse en la suma de su proyección y su diferencia o residuo: $\mathbf{x}_i = \tilde{\mathbf{x}}_i + (\mathbf{x}_i - \tilde{\mathbf{x}}_i)$. Si calculamos la norma al cuadrado de este vector desarrollando el producto escalar, obtenemos:

$$\|\mathbf{x}_i\|^2 = \|\tilde{\mathbf{x}}_i\|^2 + \|\mathbf{x}_i - \tilde{\mathbf{x}}_i\|^2 + 2\big(\tilde{\mathbf{x}}_i \cdot (\mathbf{x}_i - \tilde{\mathbf{x}}_i)\big)$$

Dado que $\tilde{\mathbf{x}}_i$ es la proyección ortogonal sobre el subespacio, el vector diferencia $(\mathbf{x}_i - \tilde{\mathbf{x}}_i)$ es perpendicular a dicho subespacio. En consecuencia, su producto escalar se anula ($\tilde{\mathbf{x}}_i \cdot (\mathbf{x}_i - \tilde{\mathbf{x}}_i) = 0$), lo que nos permite aplicar directamente el Teorema de Pitágoras:

$$\|\mathbf{x}_i\|^2 = \|\tilde{\mathbf{x}}_i\|^2 + \|\mathbf{x}_i - \tilde{\mathbf{x}}_i\|^2$$

Despejando la distancia al cuadrado $\|\mathbf{x}_i - \tilde{\mathbf{x}}_i\|^2$ y sustituyéndola en la expresión de nuestra función a minimizar E, obtenemos:

$$\frac{1}{n}\sum_{i=1}^{n}\|\mathbf{x}_i - \tilde{\mathbf{x}}_i\|^2 = \frac{1}{n}\sum_{i=1}^{n}\|\mathbf{x}_i\|^2 - \frac{1}{n}\sum_{i=1}^{n}\|\tilde{\mathbf{x}}_i\|^2$$

El primer término del lado derecho de la ecuación depende únicamente de la magnitud de los datos originales, por lo que es una constante estrictamente independiente de la elección de la base $\{\mathbf{v}_j\}$. Por lo tanto, el subespacio que minimiza la distancia a los datos originales (lado izquierdo) es aquel que maximiza el segundo término. Dado que este segundo término representa exactamente la varianza de los datos proyectados, demostramos que el subespacio óptimo es el que maximiza dicha varianza. □

Quedando establecido por qué se toma este criterio, el PCA se implementa calculando la matriz de covarianza de los datos $C = \frac{1}{n}\sum_{i=1}^{n}\mathbf{x}_i\mathbf{x}_i^T$. Pues, maximizar la varianza es equivalente a maximizar la expresión $\sum_{j=1}^{d'}\mathbf{v}_j^T C\mathbf{v}_j$ bajo la restricción de que los vectores $\{\mathbf{v}_j\}$ sean ortonormales. Este problema de optimización se resuelve usando métodos analíticos y su solución son los autovectores de la matriz de covarianza C, de los cuales se eligen los d' de mayor valor propio [27].

El PCA es solo un ejemplo de método de extracción de características, pero existen muchos otros, como los métodos basados en autoencoders, que buscan aprender una representación comprimida de los datos mediante redes neuronales [20]. Lo que todos tienen en común es la idea base de hacer uso de los datos no etiquetados para transformar la representación de los datos de entrada, con el objetivo de mejorar el rendimiento del modelo supervisado o reducir su complejidad.

5.2. Cluster-then-label

Los métodos de cluster-then-label cambian la idea anterior de realizar una transformación sobre los datos, por la idea de realizar una partición de los datos en clusters, y utilizar esta división del espacio para obtener mejoras en el rendimiento del modelo supervisado. Es decir, el *paso previo* en esta categoría consiste en aplicar un algoritmo de clustering sobre todo el conjunto de datos, independientemente de si estan etiquetados o no [28].

Desde una perspectiva formal, este enfoque transforma el problema de encontrar una única función de clasificación compleja en el problema de encontrar múltiples funciones más simples sobre subconjuntos del espacio. Sea $\mathscr{X} \subseteq \mathbb{R}^d$ el espacio de características y $\mathscr{Y}$ el espacio de etiquetas. Dado el conjunto de datos total $D = D_l \cup D_u$, el proceso se divide matemáticamente en dos fases:

1. **Fase de partición (Clustering):** Se aplica un algoritmo de aprendizaje no supervisado sobre D para encontrar la partición del espacio $\mathscr{X}$ en clústeres $\{C_1, C_2, \ldots, C_K\}$. Como resultado de este proceso, se obtiene una función de asignación $f_c : \mathscr{X} \to \{1, 2, \ldots, K\}$ que mapea cada punto de manera unívoca a su clúster correspondiente, de tal forma que $C_k = \{x \in \mathscr{X} \mid f_c(x) = k\}$.
2. **Fase de clasificación (Labeling):** En lugar de optimizar una hipótesis global $h : \mathscr{X} \to \mathscr{Y}$ en un espacio de hipótesis $\mathscr{H}$ excesivamente complejo, se restringe el problema. Para cada clúster C_k, se entrena una hipótesis local $h_k \in \mathscr{H}_k$ utilizando únicamente el subconjunto de datos etiquetados que caen dentro de esa región, es decir, el conjunto $\{(x, y) \in D_l \mid f_c(x) = k\}$.

Finalmente, el clasificador global semi-supervisado se define como una función definida a trozos (*piecewise function*), donde la predicción para una nueva instancia x viene dada por el modelo local de su clúster correspondiente:

$$h(x) = \sum_{k=1}^{K} h_k(x) \cdot \mathbb{I}(f_c(x) = k) \tag{5.2.1}$$

donde $\mathbb{I}(\cdot)$ es la función indicadora. Esta formulación permite que la estructura subyacente revelada por los datos no etiquetados (la partición) reduzca drásticamente la complejidad funcional del aprendizaje supervisado posterior.

Al igual que antes existen diferentes enfoques a la hora de aplicar esta idea que varian en función del algoritmo de clustering elegido y de la forma en la que se utiliza la partición obtenida para mejorar el rendimiento del modelo supervisado.

Un posible enfoque es el de *Multi-Manifold Semi-Supervised Learning, Goldberg et al., 2009* [12], que se basa en la idea de que los datos pueden estar distribuidos en diferentes subespacios o variedades, y que cada una de estas variedades puede ser modelada por un algoritmo de clustering diferente. En este enfoque, se aplica un algoritmo de clustering no supervisado para identificar estas variedades, y luego se entrena un modelo supervisado para cada variedad utilizando solo las muestras etiquetadas que pertenecen a dicha variedad. De esta forma, se obtiene un conjunto de modelos supervisados especializados en cada variedad, lo que puede mejorar el rendimiento del modelo global.

O el de *Semi-Supervised Clustering Using Genetic Algorithms, Demiriz et al., 1999* [9] donde se utuliza un algoritmo de clustering semi-supervisado que combina tanto la información de los datos no etiquetados como la información de las muestras etiquetadas para realizar la partición del espacio.

En general, el concepto siempre es el mismo, hacer uso de los datos no etiquetados para realizar una partición del espacio, y luego utilizar esta partición para mejorar el rendimiento del modelo supervisado. Sin embargo, las formas de implementar esta idea pueden variar mucho, y cada una de ellas puede ser más adecuada para ciertos tipos de datos o problemas específicos.

5.3. Pre-entrenamiento de modelos

Los métodos de pre-entrenamiento de modelos buscan aprovechar los datos no etiquetados para obtener una mejor inicialización de los parámetros del modelo supervisado que se va a entrenar. La idea es entrenar un modelo no supervisado utilizando los datos no etiquetados, y luego utilizar los parámetros aprendidos por este modelo como punto de partida para el entrenamiento del modelo supervisado [28].

Este tipo de métodos es especialmente útil en el contexto de las redes neuronales profundas, donde la inicialización de los parámetros puede tener un gran impacto en el rendimiento del modelo supervisado, pues los algoritmos de entrenamiento pueden converger lentamenete o presentar problemas de optimización. La idea general consiste en entrenar los parámetros de las capas inferiores de la red neuronal profunda utilizando los datos no etiquetados, y luego fine-tunear el modelo con los datos etiquetados [17, 29].

Para ilustrar este proceso desde un punto de vista matemático, el enfoque más representativo es el uso de **Autoencoders** (o codificadores automáticos) apilados. Un autoencoder es una red neuronal diseñada para aprender la función identidad, es decir, para reconstruir su propia entrada, pero forzando al modelo a aprender una representación latente útil [28].

Formalmente, dado un conjunto de datos no etiquetados $D_u = \{x_1, \ldots, x_m\}$ donde $x_i \in \mathbb{R}^d$, un autoencoder estándar se compone de dos funciones [20]:

1. **El codificador (Encoder)** h: Mapea el vector de entrada x a una representación latente oculta $z \in \mathbb{R}^{d'}$, típicamente de menor dimensión ($d' < d$). Se define como:

$$z = h(x) = \sigma(W_1 x + b_1) \tag{5.3.2}$$

   donde $W_1 \in \mathbb{R}^{d' \times d}$ es la matriz de pesos, $b_1 \in \mathbb{R}^{d'}$ es el vector sesgo, y σ es una función de activación no lineal (como sigmoide o ReLU).

2. **El decodificador (Decoder)** g: Intenta reconstruir la entrada original a partir de la representación latente. Se define como:

$$\hat{x} = g(z) = \sigma(W_2 z + b_2) \tag{5.3.3}$$

   donde $W_2 \in \mathbb{R}^{d \times d'}$ y $b_2 \in \mathbb{R}^d$.

El pre-entrenamiento no supervisado consiste en encontrar los parámetros $\theta = \{W_1, b_1, W_2, b_2\}$ que minimizan una función de pérdida que penaliza el error de reconstrucción. La función de pérdida más común es el Error Cuadrático Medio (MSE):

$$\mathscr{L}_{AE}(\theta) = \frac{1}{m}\sum_{i=1}^{m} ||x_i - g(h(x_i))||^2 \tag{5.3.4}$$

Al minimizar esta función utilizando los datos no etiquetados, los pesos W_1 y b_1 del codificador aprenden a capturar la estructura subyacente de los datos. En el contexto del pre-entrenamiento de redes profundas, una vez que este autoencoder es entrenado, se descarta el decodificador g, y los parámetros optimizados W_1 y b_1 se utilizan como la **inicialización exacta** para la primera capa oculta del modelo supervisado final. Este proceso puede repetirse secuencialmente capa por capa (Autoencoders apilados) antes de aplicar el ajuste fino (*fine-tuning*) con los datos etiquetados mediante retropropagación estándar [29].

Desde el punto de vista teórico las difentes capas de las redes neuronales profundas pueden ser vistas como diferentes niveles de abstracción en la representación de los datos, donde las capas inferiores capturan características más simples y las capas superiores capturan características más complejas. Por lo tanto, el pre-entrenamiento de modelos busca empujar al modelo hacia una mejor representación de los datos desde el principio del entrenamiento. Por lo tanto estos metodos estan intimamente relacionados con los métodos de extracción de características, aunque con la diferencia de que el pre-entrenamiento de modelos se centra en la inicialización de los parámetros del modelo supervisado que son posteriomente modificados en el entrenamietno, mientras que en los métodos de extracción de características la transformación de la representación de los datos de entrada realizada es permanente [28].

CAPÍTULO 6 Métodos intrínsecamente semi-supervisados

En este capítulo se describen los métodos intrínsecamente semi-supervisados, es decir, aquellos que desde su concepción están diseñados para entrenar de manera conjunta con datos etiquetados y no etiquetados. Su rasgo distintivo es que los ejemplos no etiquetados no aparecen como una etapa auxiliar o previa, sino que se incorporan de forma explícita en la formulación del método: bien a través de un modelo probabilístico (por ejemplo, en la verosimilitud de $p(\mathbf{x}, y)$), o bien como términos adicionales en la función objetivo (por ejemplo, regularización, consistencia o suavidad).

Siguiendo la última división de los métodos inductivos presentada en la taxonomía del Capítulo 3, organizamos los métodos intrínsecos en tres grandes grupos: métodos generativos, métodos discriminativos y métodos basados en variedades. En cada caso se introduce la intuición del enfoque y se presentan formulaciones representativas: mezclas de modelos optimizadas con EM, clasificadores de máximo margen como las S3VM y métodos basados en perturbaciones como las *ladder networks*, y, finalmente, técnicas de regularización de la variedad mediante grafos.

6.1. Generativos

Los métodos generativos constituyen uno de los enfoques clásicos del aprendizaje semi-supervisado [36]. Su idea central es modelar explícitamente el proceso que genera los datos, esto es, la distribución conjunta $p(\mathbf{x}, y)$, y, una vez estimada, usarla de diferentes formas para clasificar [28]. Este enfoque conecta directamente con la discusión del Capítulo 2 sobre cómo la información de $p(\mathbf{x})$ puede ayudar a inferir $p(y \mid \mathbf{x})$ (Sección 2.3).

Dentro de esta familia, el referente clásico son los modelos de mezcla optimizados mediante máxima verosimilitud, normalmente con el algoritmo EM cuando hay variables latentes [10].

6.1.1. Mezcla de modelos

Los modelos de mezcla asumen que los datos se generan a partir de una combinación de componentes probabilísticas, típicamente una por clase, con parámetros desconocidos [28]. Bajo este supuesto, los datos etiquetados y no etiquetados pueden contribuir conjuntamente a estimar el modelo.

La intuición en semi-supervisado es que los datos no etiquetados ayudan a identificar la estructura de la mezcla (componentes, pesos y forma), mientras que los pocos datos etiquetados permiten asociar componentes a clases [36, 35].

Mezcla de modelos supervisada

Dada una instancia $\mathbf{x}$, la predicción se obtiene tomando la clase más probable a posteriori:

$$\hat{y} = \arg\max_{y} p(y|\mathbf{x}) \tag{6.1.1}$$

Para realizar dicho cálculo, se emplea la regla de Bayes:

$$p(y|\mathbf{x}) = \frac{p(\mathbf{x}|y)p(y)}{\sum_{y'} p(\mathbf{x}|y')p(y')} \tag{6.1.2}$$

por lo que el problema se reduce a estimar $p(\mathbf{x}|y)$ y $p(y)$. En el contexto de una mezcla de componentes probabilísticos, esto equivale a estimar los parámetros que definen dichas componentes.

Si denotamos por θ al vector con los parámetros que definen los modelos y por $D_l = \{(\mathbf{x}_i, y_i)\}_{i=1}^{l}$ al conjunto etiquetado. Entonces el estimador de máxima verosimilitud de los parámetros es

$$\hat{\theta} = \arg\max_{\theta} p(D_l|\theta) = \arg\max_{\theta} \log p(D_l|\theta). \tag{6.1.3}$$

Es decir, buscamos los parámetros θ que maximizan la verosimilitud de los datos de entrenamiento. Además, como estamos tratando con datos independientes e idénticamente distribuidos, la verosimilitud se factoriza como el producto de las probabilidades individuales, lo que nos lleva a la siguiente expresión para la log-verosimilitud:

$$\log p(D_l|\theta) = \log \prod_{i=1}^{l} p(\mathbf{x}_i, y_i|\theta) = \sum_{i=1}^{l} \log\big(p(y_i|\theta)p(\mathbf{x}_i|y_i, \theta)\big). \tag{6.1.4}$$

Para familias concretas (por ejemplo, mezclas gaussianas con etiquetas observadas), este problema suele admitir estimadores cerrados o de optimización relativamente directa [35].

Mezcla de modelos semi-supervisada

En el caso semi-supervisado se dispone, además de D_l, de un conjunto no etiquetado $D_u = \{\mathbf{x}_j\}_{j=l+1}^{l+u}$. El conjunto completo es $D = D_l \cup D_u$, y la log-verosimilitud pasa a ser:

$$\log p(D|\theta) = \sum_{i=1}^{l} \log p(\mathbf{x}_i, y_i|\theta) + \sum_{j=l+1}^{l+u} \log p(\mathbf{x}_j|\theta). \tag{6.1.5}$$

Donde en el término de los datos no etiquetados se incorpora la distribución marginal

$$p(\mathbf{x}_j|\theta) = \sum_{y \in \mathscr{Y}} p(\mathbf{x}_j, y|\theta) = \sum_{y \in \mathscr{Y}} p(y|\theta)p(\mathbf{x}_j|y, \theta), \tag{6.1.6}$$

que refleja que la etiqueta de $\mathbf{x}_j$ es desconocida [35].

En general, esta optimización deja de ser cóncava y no suele resolverse de forma analítica. Por ello, el método estándar es EM, un procedimiento iterativo de máxima verosimilitud con variables latentes [10]. De forma intuitiva, EM repite el siguiente ciclo:

- **Inicialización**: partir de unos parámetros iniciales $\theta^{(0)}$ (por ejemplo, obtenidos con los datos etiquetados).
- **E-step**: con los parámetros actuales, estimar para cada dato no etiquetado la probabilidad de pertenecer a cada clase, es decir, una etiqueta *blanda*.
- **M-step**: de manera análoga al caso supervisado, reestimar los parámetros del modelo, pero usando ahora las etiquetas probabilísticas obtenidas en la E-step como pesos en la máxima verosimilitud.
- **Iteración**: repetir E-step y M-step hasta convergencia.

Esta dinámica es conceptualmente parecida al pseudoetiquetado de los métodos envolventes (Capítulo 4) [35, 34], pero aquí las asignaciones para D_u son probabilísticas, mientras que en los métodos envolventes se termina obteniendo una etiqueta final para cada ejemplo.

La mezcla de modelos es un enfoque que puede ser muy eficaz cuando el modelo generativo está bien especificado. Sin embargo, su rendimiento depende críticamente de hipótesis como la identificabilidad de la mezcla y la corrección del modelo; si estas fallan, los datos no etiquetados pueden degradar el rendimiento [36, 28].

6.2. Discriminativos

Esta familia de métodos se distingue de la anterior en que no trata de modelar el proceso generativo ni la densidad conjunta $p(\mathbf{x}, y)$, sino que se centra directamente en aprender un modelo discriminativo $p(y \mid \mathbf{x})$ o una función de decisión $f : \mathscr{X} \to \mathscr{Y}$ [23]. En este caso, la información de los datos no etiquetados se incorpora con el objetivo de mejorar la generalización del modelo, normalmente a través de suposiciones adicionales sobre la distribución subyacente de los datos (por ejemplo, la separación en baja densidad o la suposición de suavidad) [28].

6.2.1. Máximo margen

Los métodos de máximo margen buscan aprender una frontera de decisión que quede lo más alejada posible de los ejemplos de entrenamiento. En el caso de clasificación binaria, esto suele formularse como la maximización del margen geométrico del clasificador [28]. En aprendizaje semi-supervisado, esta idea se combina con la suposición de separación en baja densidad: además de clasificar bien los ejemplos etiquetados, se prefiere que la frontera atraviese regiones del espacio de entrada donde haya pocos datos [28, 12].

El representante más conocido es la *Semi-Supervised Support Vector Machine* (S3VM), también denominada históricamente *Transductive SVM* (TSVM) [8]. Para introducirla, repasamos primero la formulación básica de las SVM.

SVM

Consideramos un conjunto etiquetado $\{(\mathbf{x}_i, y_i)\}_{i=1}^{l}$ con $\mathbf{x}_i \in \mathbb{R}^d$ e $y_i \in \{-1, 1\}$, y un clasificador lineal de la forma

$$f(\mathbf{x}) = \mathbf{w}^\top \mathbf{x} + b, \tag{6.2.7}$$

donde $\mathbf{w} \in \mathbb{R}^d$ y $b \in \mathbb{R}$ [35]. Por lo que la frontera de decisión es el siguiente hiperplano:

$$\left\{\mathbf{x} \in \mathbb{R}^d : \mathbf{w}^\top \mathbf{x} + b = 0\right\}, \tag{6.2.8}$$

mientras que la predicción se obtiene con $\operatorname{sign}(f(\mathbf{x}))$.

La distancia (no firmada) de un punto $\mathbf{x}$ a dicho hiperplano es $\frac{|f(\mathbf{x})|}{\|\mathbf{w}\|}$, y para los ejemplos etiquetados se toma la distancia con signo que es $\frac{y_i f(\mathbf{x}_i)}{\|\mathbf{w}\|}$. Con todo esto, el margen geométrico del clasificador se define como

$$\gamma = \min_{i=1,\dots,l} \frac{y_i\left(\mathbf{w}^\top \mathbf{x}_i + b\right)}{\|\mathbf{w}\|}. \tag{6.2.9}$$

Primero, vamos a asumir que los datos son *linealmente separables*, es decir, que existe un hiperplano tal que $y_i\left(\mathbf{w}^\top \mathbf{x}_i + b\right) > 0$ para todo $i = 1, \dots, l$. Bajo esta suposición, el *hard-margin SVM* busca el hiperplano separador que maximiza γ, lo que puede escribirse como

$$\max_{\mathbf{w},b} \quad \min_{i=1,\dots,l} \frac{y_i\left(\mathbf{w}^\top \mathbf{x}_i + b\right)}{\|\mathbf{w}\|}. \tag{6.2.10}$$

Este problema es invariante frente a escalado: si multiplicamos $(\mathbf{w}, b)$ por una constante $c > 0$, la frontera de decisión $\{\mathbf{x} : \mathbf{w}^\top \mathbf{x} + b = 0\}$ no cambia. Por ello, podemos fijar una escala imponiendo $\min_i y_i\left(\mathbf{w}^\top \mathbf{x}_i + b\right) = 1$, equivalente a exigir $y_i\left(\mathbf{w}^\top \mathbf{x}_i + b\right) \geq 1$ para todo i. En consecuencia, maximizar el margen es equivalente a maximizar $\frac{1}{\|\mathbf{w}\|}$, o de forma equivalente minimizar la norma de $\mathbf{w}$:

$$\begin{aligned} \min_{\mathbf{w},b} \quad & \frac{1}{2}\|\mathbf{w}\|^2 \\ \text{s.a.} \quad & y_i\left(\mathbf{w}^\top \mathbf{x}_i + b\right) \geq 1, \quad i = 1, \dots, l. \end{aligned} \tag{6.2.11}$$

Cuando los datos no son separables (esto es, no existe un hiperplano que separe perfectamente las clases), se introduce el *soft margin* mediante variables de holgura $\xi_i \geq 0$ y un parámetro $C > 0$ que controla el compromiso entre maximizar el margen y minimizar errores. Intuitivamente, ξ_i mide cuánto viola el ejemplo i la restricción de margen: $\xi_i = 0$ significa que se cumple el margen, $0 < \xi_i < 1$ indica que el punto queda dentro del margen pero sigue estando bien clasificado, y $\xi_i > 1$ corresponde a una clasificación errónea.

$$\begin{aligned} \min_{\mathbf{w},b} \quad & \frac{1}{2}\|\mathbf{w}\|^2 + C\sum_{i=1}^{l} \xi_i \\ \text{s.a.} \quad & y_i\left(\mathbf{w}^\top \mathbf{x}_i + b\right) \geq 1 - \xi_i, \quad i = 1, \dots, l, \\ & \xi_i \geq 0. \end{aligned} \tag{6.2.12}$$

Finalmente, puede expresarse en el marco de la minimización de riesgo regularizado (Sección 2.2). Despejando ξ_i de $y_i f(\mathbf{x}_i) \geq 1 - \xi_i$ obtenemos $\xi_i \geq 1 - y_i f(\mathbf{x}_i)$, y junto con $\xi_i \geq 0$ se tiene $\xi_i = \max\left(0, 1 - y_i f(\mathbf{x}_i)\right)$. Por lo tanto, la función objetivo se reescribe como

$$\min_{\mathbf{w},b} \quad \frac{1}{2}\|\mathbf{w}\|^2 + C\sum_{i=1}^{l} \max\left(0, 1 - y_i f(\mathbf{x}_i)\right). \tag{6.2.13}$$

En este caso la función de pérdida es $c(\mathbf{x}, y, f(\mathbf{x})) = \max\left(0, 1 - y f(\mathbf{x})\right)$ (pérdida *hinge*) y el regularizador es $\frac{1}{2}\|\mathbf{w}\|^2$.

S3VM

La idea central de las S3VM es incorporar un conjunto no etiquetado $\{\mathbf{x}_j\}_{j=l+1}^{l+u}$ penalizando que dichos puntos queden cerca de la frontera: se prefiere que $|f(\mathbf{x}_j)|$ sea grande, de modo que la frontera se sitúe en regiones de baja densidad [28, 8]. Una forma estándar de derivar esta penalización es introducir etiquetas latentes $\hat{y}_j \in \{-1,1\}$ para los ejemplos no etiquetados y añadir un término de pérdida *hinge* $\operatorname{máx}\big(0, 1-\hat{y}_j f(\mathbf{x}_j)\big)$, que al minimizar respecto a $\hat{y}_j$ se convierte en $\operatorname{máx}\big(0, 1-|f(\mathbf{x}_j)|\big)$, conocida como *hat loss* [8].

Con ello, una formulación típica de S3VM es

$$\min_{\mathbf{w},b} \quad \frac{1}{2}\|\mathbf{w}\|^2 + C\sum_{i=1}^{l} \operatorname{máx}\big(0, 1-y_i f(\mathbf{x}_i)\big) + C^* \sum_{j=l+1}^{l+u} \operatorname{máx}\big(0, 1-|f(\mathbf{x}_j)|\big). \tag{6.2.14}$$

En términos del marco de la Sección 2.2, el sumatorio sobre los datos no etiquetados puede interpretarse como un término adicional dependiente de $\{\mathbf{x}_j\}$, que incorpora explícitamente la suposición de separación en baja densidad.

En la práctica suele ser necesario imponer alguna restricción adicional (por ejemplo, sobre el balance de etiquetas asignadas a los datos no etiquetados) para evitar soluciones degeneradas [28]. El problema resultante es no convexo, por lo que se recurre a métodos heurísticos y aproximados; además, si la suposición de separación en baja densidad no se cumple, el uso de la información no etiquetada puede incluso degradar el rendimiento [28].

6.2.2. Basados en perturbaciones

Por otra parte, los métodos basados en perturbaciones se fundamentan en la suposición de suavidad: puntos cercanos en el espacio de entrada deberían tener etiquetas similares [8]. Para aprovechar esta suposición, se introducen perturbaciones o ruido estocástico en los datos de entrenamiento y se añade un término de regularización por consistencia a la función de pérdida. Este término penaliza que el modelo produzca predicciones diferentes para las versiones original y perturbada de un mismo ejemplo [28].

Este enfoque se ha popularizado especialmente en el contexto de las redes neuronales profundas, debido a su capacidad para aprender representaciones ricas y a la facilidad algorítmica de incorporar perturbaciones en el proceso de entrenamiento [28]. Un ejemplo pionero es el método *ladder network* [24], que combina una arquitectura de red neuronal con un término de pérdida que evalúa la discrepancia entre las activaciones de la red para ejemplos limpios y perturbados.

Redes neuronales

Antes de proceder a exponer el método de las *ladder networks*, repasamos brevemente el marco general de las redes neuronales, más concretamente el marco de las redes neuronales feedforward.

Una red neuronal es un sistema que transforma un vector de entrada $\mathbf{x} \in \mathbb{R}^d$ en un vector de salida $f(\mathbf{x}) \in \mathbb{R}^k$ mediante la propagación de la entrada por una serie de capas conectadas secuencialmente [28].

Cada capa de la red se compone de un conjunto de perceptrones, un modelo matemático de neurona. Cada uno de estos perceptrones recibe como entrada un vector $\mathbf{z} \in \mathbb{R}^m$, o bien el de entrada o bien las salidas de la capa anterior, y produce una salida escalar $a \in \mathbb{R}$ mediante la aplicación de una función de activación $\sigma : \mathbb{R} \to \mathbb{R}$ a una combinación lineal de las entradas, es decir, $a = \sigma(\mathbf{w}^\top \mathbf{z} + b)$, donde $\mathbf{w} \in \mathbb{R}^m$ y $b \in \mathbb{R}$ son los parámetros del perceptrón [6].

En el contexto de aprendizaje supervisado, el entrenamiento de la red neuronal consiste en ajustar los parámetros de los perceptrones, representados por una matriz W que contiene tanto los pesos $\mathbf{w}$ como los sesgos b, para minimizar una función de pérdida [28].

$$\mathscr{L}(W) = \frac{1}{l} \sum_{i=1}^{l} c(\mathbf{x}_i, y_i, f(\mathbf{x}_i; W)) \tag{6.2.15}$$

Los parámetros se optimizan iterativamente mediante la retropropagación del error, normalmente con un método de descenso de gradiente estocástico o alguna de sus variantes [14].

Ladder networks

Una vez introducidos los conceptos básicos de las redes neuronales, es posible presentar el método de las *ladder networks* [24]. Este enfoque extiende la arquitectura de una red neuronal feedforward estándar incorporando un término de pérdida no supervisado, basado en el principio de los autoencoders eliminadores de ruido.

En concreto, el modelo se estructura en un codificador y un decodificador. Durante el entrenamiento, cada dato de entrada realiza dos pasadas por el codificador: una pasada limpia y una pasada a la que se le inyecta ruido en todas las capas ocultas. Posteriormente, el decodificador intenta ir limpiando este ruido hacia atrás, reconstruyendo las activaciones limpias originales capa por capa a partir de las representaciones perturbadas. De este modo, el término de pérdida global se define como la suma del coste de clasificación supervisado y un coste de consistencia o reconstrucción calculado a lo largo de todas las capas de la red [24].

La función de pérdida total a optimizar se puede formular de la siguiente manera:

$$\mathscr{L}(W) = \frac{1}{l} \sum_{i=1}^{l} c(\mathbf{x}_i, y_i, f(\mathbf{x}_i; W)) + \sum_{i=1}^{l+u} \sum_{k=1}^{K} ReconstructionLoss\left(\hat{\mathbf{z}}_i^k, \mathbf{z}_i^k\right) \tag{6.2.16}$$

Donde $\mathbf{z}_i^k$ es la activación oculta de la capa k en el codificador para el ejemplo limpio $\mathbf{x}_i$, mientras que $\hat{\mathbf{z}}_i^k$ es la activación reconstruida por el decodificador a partir de la versión perturbada. Finalmente, ReconstructionLoss es una función que penaliza la discrepancia entre ambas (como, por ejemplo, la norma L_2 al cuadrado) [24].

A través de esta arquitectura, las *ladder networks* consiguen aprovechar simultáneamente tanto la señal de los datos etiquetados como la regularización proporcionada por los datos no etiquetados[1], forzando a la red a extraer características latentes útiles y robustas al ruido que, en última instancia, mejoran la generalización del modelo predictivo [24].

[1] Esta simultaneidad es, de hecho, lo que hace a las *ladder networks* modelos intrínsecos, a diferencia del preentrenamiento por capas visto en la Sección 5.3, donde se usaba un *encoder-decoder* como fase inicial independiente seguida de la fase supervisada.

6.3. Basados en variedades

Finalmente, los métodos basados en variedades constituyen una categoría independiente en la presente taxonomía. Por un lado, no pueden considerarse generativos, puesto que no modelan explícitamente el proceso de generación de los datos. Por otro lado, aunque (al igual que los métodos discriminativos) se centran en aprender una función de decisión, su enfoque subyacente es diferente al de las técnicas de máximo margen o de perturbaciones.

Estos métodos se apoyan en la hipótesis de la variedad: se asume que los datos observados se concentran cerca de una variedad de baja dimensión inmersa en el espacio de entrada original [8]. En consecuencia, el papel de los datos no etiquetados no es solo ajustar una frontera global, sino capturar la geometría de dicha variedad y orientar la búsqueda hacia funciones que respeten su estructura intrínseca [28].

6.3.1. Técnicas de regularización de la variedad

Una forma habitual de aproximar la variedad subyacente es construir un grafo de similitud entre instancias [28]. Dado un conjunto de entrenamiento formado por $\{(\mathbf{x}_i, y_i)\}_{i=1}^{l}$ instancias etiquetadas y $\{\mathbf{x}_j\}_{j=l+1}^{l+u}$ instancias sin etiquetar, se construye un grafo no dirigido $G = (V, E)$ donde cada nodo corresponde a una instancia (etiquetada o no)[2], y a cada arista $(i, j) \in E$ se le asigna un peso w_{ij} que refleja la similitud entre $\mathbf{x}_i$ y $\mathbf{x}_j$. Denotamos por $W = (w_{ij})$ la matriz de pesos. La intuición es que, si w_{ij} es alto, entonces es más probable que ambas instancias compartan etiqueta [35].

Hay varias formas de definir la vecindad y/o la similitud. Por ejemplo, puede construirse un grafo de k vecinos más cercanos (k-NN) y asignar pesos gaussianos $w_{ij} = \exp\left(-\frac{\|\mathbf{x}_i - \mathbf{x}_j\|^2}{2\sigma^2}\right)$ a las aristas [35].

Una vez construido el grafo, se busca una función de decisión $f : \mathscr{X} \to \mathscr{Y}$ que no solo clasifique correctamente los ejemplos etiquetados, sino que también sea suave respecto a la estructura del grafo; es decir, que produzca salidas similares en nodos conectados por aristas con pesos altos [35].

Para ello, un enfoque representativo es el presentado en *On Manifold Regularization, Belkin et al., 2005* [4]. Partiendo de un marco de aprendizaje supervisado, se considera un kernel K[3], con un espacio de funciones asociado $\mathscr{H}_K$ y norma $\|\cdot\|_K$. Se busca una función $f \in \mathscr{H}_K$ que minimice la siguiente función de pérdida regularizada:

$$\mathscr{L}(f) = \frac{1}{l}\sum_{i=1}^{l} c(\mathbf{x}_i, y_i, f(\mathbf{x}_i)) + \lambda \|f\|_K^2, \tag{6.3.17}$$

donde el primer término es la pérdida de clasificación sobre los datos etiquetados y el segundo término es un regularizador que penaliza la complejidad de la función f [4].

Finalmente, la información no etiquetada se introduce mediante un término de regularización adicional, utilizando la matriz de pesos W, que penaliza la falta de suavidad de f respecto

[2]Como consecuencia, los grafos resultantes suelen tener gran tamaño cuando se dispone de una gran cantidad de instancias no etiquetadas [35].

[3]Un kernel es una función $K : \mathscr{X} \times \mathscr{X} \to \mathbb{R}$ que puede interpretarse como un producto escalar en un espacio de características (posiblemente de mayor dimensión) [6].

a la estructura del grafo:

$$\|f\|_I^2 = \frac{1}{2}\sum_{i,j} w_{ij}\big(f(\mathbf{x}_i) - f(\mathbf{x}_j)\big)^2, \tag{6.3.18}$$

Este término puede interpretarse como una medida de la variación de f a lo largo de las aristas del grafo, ponderada por los pesos de similitud. Además, si denotamos $n = l + u$ y definimos el vector $\mathbf{f} = \big(f(\mathbf{x}_1), \ldots, f(\mathbf{x}_n)\big)^\top$, la matriz diagonal de grados D con $D_{ii} = \sum_j w_{ij}$ y el laplaciano no normalizado $L = D - W$, entonces se obtiene una expresión matricial compacta para él mismo $\|f\|_I^2 = \mathbf{f}^\top L\mathbf{f}$ [4].

De este modo, la función de pérdida total a minimizar es

$$\mathscr{L}(f) = \frac{1}{l}\sum_{i=1}^{l} c(\mathbf{x}_i, y_i, f(\mathbf{x}_i)) + \lambda_A \|f\|_K^2 + \lambda_I \|f\|_I^2, \tag{6.3.19}$$

donde λ_A y λ_I son hiperparámetros que controlan el peso relativo de cada término de regularización [4].

Dependiendo de la elección del kernel K y de la función de pérdida c, esta formulación puede dar lugar a diferentes algoritmos concretos. Un ejemplo clásico son las *Laplacian SVM*, que combinan una pérdida *hinge* con el término de regularización basado en el grafo. De este modo, a diferencia de las S3VM vistas en la sección anterior, la información no etiquetada se incorpora a través de la geometría del grafo y no mediante la penalización directa de puntos cercanos a la frontera de decisión [28].

En la práctica, estos métodos pueden ser costosos computacionalmente, ya que suelen requerir operar con matrices de tamaño $n \times n$ (donde $n = l + u$). En particular, en algunos casos el coste puede alcanzar $O(n^3)$, o bien $O(cn^2)$ para un $c \ll n$ [28]. Además, el rendimiento es muy sensible a cómo se construye el grafo y a la elección y escala de los pesos w_{ij}; si el grafo no refleja bien la geometría de los datos, la información no etiquetada puede propagarse de forma perjudicial [35].

CAPÍTULO 7

Métodos Transductivos

Hasta ahora, las familias de métodos presentadas en los capítulos anteriores eran *inductivas*: su objetivo era construir un modelo a partir de los datos de entrenamiento y, posteriormente, usarlo para predecir la clase de nuevas instancias del espacio de entrada. En este capítulo se presentan métodos *transductivos*, que en lugar de aprender una regla de decisión general, se centran en inferir la clase de las instancias no etiquetadas disponibles en el propio conjunto de entrenamiento [28].

El enfoque transductivo aparece de manera natural en los métodos semi-supervisados basados en grafos. En ellos se construye un grafo de similitud donde cada nodo representa una instancia (etiquetada o no) y las aristas ponderadas codifican la proximidad entre pares de ejemplos. Estos métodos suelen asumir *suavidad de etiquetas* sobre el grafo: si dos nodos están conectados por una arista con peso grande, entonces es razonable esperar que sus etiquetas (o puntuaciones de clase) sean similares [36].

Esta idea es conceptualmente cercana a la de los métodos basados en variedades (véase la Sección 6.3), donde el grafo se utiliza como una aproximación discreta de la geometría de los datos. La diferencia clave es que, en el planteamiento transductivo, la función de decisión se define *directamente sobre los nodos* del grafo, de modo que el problema se reduce a propagar de forma coherente la información de las etiquetas conocidas hacia los nodos no etiquetados [28].

Usando el marco de pérdida y regularización introducido en la Sección 2.2, el aprendizaje transductivo sobre grafos puede formularse como la minimización de un término de ajuste en los nodos etiquetados más un término que penaliza la falta de suavidad sobre el grafo [28, 36].

$$\min_{f} \sum_{i \in L} \ell\big(f(\mathbf{x}_i), y_i\big) + \lambda \sum_{i,j} w_{ij}\, \ell_U\big(f(\mathbf{x}_i), f(\mathbf{x}_j)\big) \tag{7.0.1}$$

En esta expresión, L denota el conjunto de índices etiquetados y U el de no etiquetados (con $L \cup U = \{1, \ldots, n\}$). Además, $w_{ij} \geq 0$ cuantifica la similitud entre $\mathbf{x}_i$ y $\mathbf{x}_j$ (y, por convenio, $w_{ij} = 0$ cuando no existe arista entre ambos nodos), mientras que ℓ_U es una penalización que favorece predicciones similares en nodos conectados.

Observación 7.0.1 Nótese la similitud con el enfoque visto en los métodos de regularización de variedades. La diferencia aquí es que, en el planteamiento puramente transductivo, no se busca controlar la complejidad de un modelo definido sobre todo el espacio de entrada, sino obtener predicciones coherentes para las instancias del conjunto de entrenamiento; si se desea extrapolar a ejemplos nuevos, se pasa a formulaciones inductivas como las de la Sección 6.3.

Por su propia estructura, estos métodos tienen dos partes claramente diferenciadas: la **construcción del grafo** de similitud (definir vecindades y pesos) y la **inferencia sobre el grafo** (obtener la asignación de etiquetas o la función f en los nodos) [28]. Pasemos entonces a abordar cada una de estas partes.

7.1. Construcción del grafo

La construcción del grafo constituye la fase inicial de los métodos basados en grafos y, en la práctica, suele ser un componente determinante: la calidad del grafo de similitud condiciona de manera directa el rendimiento del método de inferencia posterior [28]. Dado que los nodos representan instancias del conjunto de entrenamiento, construir el grafo equivale a decidir (i) qué pares de nodos se conectan mediante aristas y (ii) qué peso (similitud) se asigna a cada una de ellas.

7.1.1. Construcción de la matriz de adyacencia

El primer paso de la construcción de un grafo de similitud es definir una matriz de adyacencia A, donde cada elemento a_{ij} indica la existencia de una arista entre los nodos i y j. Existen varias formas de construir esta matriz, siendo las más comunes:

- **Grafo completo**: se conecta cada par de nodos, es decir, $a_{ij} = 1$ para todo $i \neq j$ [35]. Aunque conceptualmente sencillo, suele ser impracticable por su coste computacional y rara vez se utiliza en problemas de tamaño medio o grande.
- **Grafo de ε-vecinos**: se conecta el nodo i con el nodo j si la distancia entre sus representaciones es menor que un umbral ε, esto es, $a_{ij} = 1$ si $d(\mathbf{x}_i, \mathbf{x}_j) < \varepsilon$. Es un criterio simple, pero el grafo resultante depende fuertemente de la elección de ε y puede ser sensible a la escala de los datos [28].
- **Grafo de k vecinos más cercanos (k-NN)**: se conecta el nodo i con el nodo j si j pertenece al conjunto de los k vecinos más cercanos de i. En general, este procedimiento produce un grafo dirigido (puede ocurrir que j sea vecino de i sin que i lo sea de j). Para obtener un grafo no dirigido se suele *simetrizar* la vecindad, por ejemplo conectando i y j si i es vecino de j *o* j es vecino de i, o bien exigiendo que la vecindad sea recíproca. Es uno de los enfoques más utilizados en la práctica, aunque también requiere seleccionar el parámetro k [28].
- **Grafo mediante *b-matching***: mientras que los criterios anteriores son esencialmente locales, el *b-matching* busca construir conexiones optimizando un objetivo global [28]. De forma informal, se selecciona un conjunto de aristas que maximiza una medida de similitud agregada, sujeto a que cada nodo tenga a lo sumo b conexiones. Puede producir grafos menos sensibles a parámetros locales, a costa de una implementación más compleja y un mayor coste computacional [19].

7.1.2. Asignación de pesos

Una vez definida la estructura del grafo, el siguiente paso es asignar pesos a las aristas. Estos pesos reflejan la similitud entre los nodos conectados, y su elección puede afectar significativa-

mente el rendimiento del método de inferencia [28]. Algunas formas comunes de asignar pesos son:

- **Asignación gaussiana**: Una forma común de asignar pesos es usar una función gaussiana de la distancia entre los nodos, es decir,

$$w_{ij} = \exp(-\frac{\|\mathbf{x}_i - \mathbf{x}_j\|^2}{2\sigma^2}) \tag{7.1.2}$$

  donde σ es un parámetro que controla la velocidad de decaimiento de la similitud [35]. Este método es popular porque produce pesos que decaen suavemente con la distancia, pero requiere elegir el parámetro σ de manera adecuada.

- **Asignación gaussiana modificada**: Una variante de la anterior es usar una función gaussiana modificada que tenga en cuenta la densidad local de los datos, es decir,

$$w_{ij} = \exp(-\frac{\|\mathbf{x}_i - \mathbf{x}_j\|^2}{\text{máx}\{\sigma_i, \sigma_j\}^2}) \tag{7.1.3}$$

  donde σ_i y σ_j son las distancias a los vecinos más cercanos de los nodos i y j, respectivamente [16].

- **Propagación lineal de vecinos**: Los métodos anteriores asignan w_{ij} a partir de una función explícita de la distancia entre dos nodos. Sin embargo, existen alternativas que incorporan información de todo el vecindario. Un ejemplo es asignar pesos imponiendo que cada instancia pueda aproximarse como una combinación lineal de sus vecinos, minimizando un error de reconstrucción [28]:

$$\min_{\{w_{ij}\}} \sum_i \left\| \mathbf{x}_i - \sum_{j \in N(i)} w_{ij} \mathbf{x}_j \right\|^2 \tag{7.1.4}$$

  sujeto a la restricción $\sum_{j \in N(i)} w_{ij} = 1$ para cada nodo i, donde $N(i)$ denota el conjunto de vecinos de i en el grafo. Este enfoque puede producir pesos mejor adaptados a la estructura local de los datos, aunque conlleva un mayor coste computacional [28].

7.2. Inferencia sobre el grafo

La construcción del grafo descrita en la sección anterior es necesaria siempre que se quiera explotar una noción de vecindad o geometría sobre el conjunto de entrenamiento; por lo que, también es un ingrediente central en los métodos de regularización de variedades (Sección 6.3). La parte distintiva del enfoque transductivo es la **inferencia sobre el grafo**: dado el grafo de similitud y las etiquetas de un subconjunto de nodos, se trata de asignar etiquetas (o, más generalmente, una función de decisión) a los nodos no etiquetados, respetando la suavidad inducida por los pesos w_{ij}.

Una vez fijado el grafo, muchos métodos de inferencia pueden interpretarse como instancias del problema de optimización de la Ecuación 7.0.1, que combina (i) un término de ajuste a las etiquetas conocidas y (ii) un término de regularización que penaliza variaciones bruscas de f entre nodos similares. La elección de las pérdidas ℓ y ℓ_U, del parámetro λ y del algoritmo para resolver el problema determina el método concreto [28]. A continuación se presentan dos enfoques clásicos.

7.2.1. Mincut

El método *mincut* (o *s-t mincut*) se plantea típicamente para clasificación binaria y busca un corte del grafo que separe los nodos etiquetados positivos de los negativos minimizando la suma de pesos de las aristas cortadas [35]. Una vez fijado el corte, los nodos no etiquetados heredan la etiqueta del lado del corte al que quedan conectados.

Atendiendo a la formulación de la Ecuación 7.0.1, este método puede verse como la elección de una pérdida supervisada con peso infinito (equivalentemente, imponiendo como restricciones las etiquetas en L) junto con un término de suavidad cuadrático en el grafo. Una formulación habitual es

$$\min_{f:f(\mathbf{x}_i)\in\{-1,1\}} \frac{1}{4}\sum_{i,j} w_{ij}\left(f(\mathbf{x}_i)-f(\mathbf{x}_j)\right)^2 \quad \text{s.a.} \quad f(\mathbf{x}_i)=y_i,\ i\in L. \tag{7.2.5}$$

El enfoque tiene limitaciones conocidas: en su forma básica está restringido a clasificación binaria y puede presentar cortes degenerados o no únicos, lo que se traduce en soluciones inestables o poco informativas para los nodos no etiquetados [35, 28].

7.2.2. Funciones armónicas

La idea de este método es relajar el problema de *mincut* permitiendo que, en los nodos no etiquetados, la función f tome valores continuos (por ejemplo, puntuaciones en $\mathbb{R}$), manteniendo las etiquetas conocidas fijadas en L. Gracias a esta relajación, el problema se reduce a la minimización de una función cuadrática, lo que conduce a una solución cerrada[1] que, bajo condiciones suaves, es única y óptima [35].

El nombre proviene de la conexión con las funciones armónicas: la solución satisface una condición de *media ponderada* sobre el grafo. En particular, para cada nodo no etiquetado $i \in U$ se cumple [28]:

$$\sum_{j\in N(i)} w_{ij}(f(\mathbf{x}_i)-f(\mathbf{x}_j))=0 \tag{7.2.6}$$

equivalentemente, $f(\mathbf{x}_i)$ coincide con el promedio ponderado de sus vecinos, con pesos proporcionales a w_{ij}.

[1] Es decir, tiene una forma explícita que se calcula resolviendo un sistema de ecuaciones lineales

Bibliografía

[1] Hervé Abdi and Lynne J Williams. Principal component analysis. *Wiley interdisciplinary reviews: computational statistics*, 2(4):433–459, 2010.

[2] Steven Abney. Bootstrapping. In *Proceedings of the 40th annual meeting of the Association for Computational Linguistics*, pages 360–367, 2002.

[3] Maria-Florina Balcan, Avrim Blum, and Ke Yang. Co-training and expansion: Towards bridging theory and practice. *Advances in neural information processing systems*, 17, 2004.

[4] Misha Belkin, Partha Niyogi, and Vikas Sindhwani. On manifold regularization. In *International Workshop on Artificial Intelligence and Statistics*, pages 17–24. PMLR, 2005.

[5] Kristin P Bennett, Ayhan Demiriz, and Richard Maclin. Exploiting unlabeled data in ensemble methods. In *Proceedings of the eighth ACM SIGKDD international conference on Knowledge discovery and data mining*, pages 289–296, 2002.

[6] Christopher M Bishop and Nasser M Nasrabadi. *Pattern recognition and machine learning*, volume 4. Springer, 2006.

[7] Avrim Blum and Tom Mitchell. Combining labeled and unlabeled data with co-training. In *Proceedings of the eleventh annual conference on Computational learning theory*, pages 92–100, 1998.

[8] Olivier Chapelle, Bernhard Schölkopf, and Alexander Zien. *Semi-Supervised Learning*. The MIT Press, 09 2006.

[9] Ayhan Demiriz, Kristin P Bennett, and Mark J Embrechts. Semi-supervised clustering using genetic algorithms. *Artificial neural networks in engineering (ANNIE-99)*, pages 809–814, 1999.

[10] Arthur P Dempster, Nan M Laird, and Donald B Rubin. Maximum likelihood from incomplete data via the em algorithm. *Journal of the royal statistical society: series B (methodological)*, 39(1):1–22, 1977.

[11] Vladimir Estivill-Castro. Why so many clustering algorithms: a position paper. *ACM SIGKDD explorations newsletter*, 4(1):65–75, 2002.

[12] Andrew Goldberg, Xiaojin Zhu, Aarti Singh, Zhiting Xu, and Robert Nowak. Multi-manifold semi-supervised learning. In David van Dyk and Max Welling, editors, *Proceedings of the Twelfth International Conference on Artificial Intelligence and Statistics*, volume 5 of *Proceedings of Machine Learning Research*, pages 169–176, Hilton Clearwater Beach Resort, Clearwater Beach, Florida USA, 16–18 Apr 2009. PMLR.

[13] Sally A. Goldman and Yan Zhou. Enhancing supervised learning with unlabeled data. In *International Conference on Machine Learning*, 2000.

[14] Ian J. Goodfellow, Jean Pouget-Abadie, Mehdi Mirza, Bing Xu, David Warde-Farley, Sherjil Ozair, Aaron Courville, and Yoshua Bengio. Generative adversarial nets. In Z. Ghahramani, M. Welling, C. Cortes, N. Lawrence, and K.Q. Weinberger, editors, *Advances in Neural Information Processing Systems*, volume 27. Curran Associates, Inc., 2014.

[15] Y Grandvalet, F d'Alché Buc, and C Ambroise. Boosting mixture models for semi-supervised learning, vol. 2130. 2001.

[16] Matthias Hein and Markus Maier. Manifold denoising. *Advances in neural information processing systems*, 19, 2006.

[17] Geoffrey E Hinton, Simon Osindero, and Yee-Whye Teh. A fast learning algorithm for deep belief nets. *Neural computation*, 18(7):1527–1554, 2006.

[18] Anil K Jain and Richard C Dubes. *Algorithms for clustering data*. Prentice-Hall, Inc., 1988.

[19] Tony Jebara, Jun Wang, and Shih-Fu Chang. Graph construction and b-matching for semi-supervised learning. In *Proceedings of the 26th annual international conference on machine learning*, pages 441–448, 2009.

[20] Pengzhi Li, Yan Pei, and Jianqiang Li. A comprehensive survey on design and application of autoencoder in deep learning. *Applied Soft Computing*, 138:110176, 2023.

[21] Pavan Kumar Mallapragada, Rong Jin, Anil K Jain, and Yi Liu. Semiboost: Boosting for semi-supervised learning. *IEEE transactions on pattern analysis and machine intelligence*, 31(11):2000–2014, 2008.

[22] Mehryar Mohri, Afshin Rostamizadeh, and Ameet Talwalkar. *Foundations of machine learning*. MIT press, 2018.

[23] Andrew Ng and Michael Jordan. On discriminative vs. generative classifiers: A comparison of logistic regression and naive bayes. *Advances in neural information processing systems*, 14, 2001.

[24] Antti Rasmus, Mathias Berglund, Mikko Honkala, Harri Valpola, and Tapani Raiko. Semi-supervised learning with ladder networks. *Advances in neural information processing systems*, 28, 2015.

[25] Matthias W Seeger et al. A taxonomy for semi-supervised learning methods., 2006.

[26] Vikas Sindhwani and David S Rosenberg. An rkhs for multi-view learning and manifold co-regularization. In *Proceedings of the 25th international conference on Machine learning*, pages 976–983, 2008.

[27] A Guided Tour. Geometric methods for feature extraction and dimensional reduction. *Data Mining and Knowledge Discovery Handbook*, page 59, 2006.

[28] Jesper E Van Engelen and Holger H Hoos. A survey on semi-supervised learning. *Machine learning*, 109(2):373–440, 2020.

[29] Pascal Vincent, Hugo Larochelle, Yoshua Bengio, and Pierre-Antoine Manzagol. Extracting and composing robust features with denoising autoencoders. In *Proceedings of the 25th international conference on Machine learning*, pages 1096–1103, 2008.

[30] Jiao Wang, Si-wei Luo, and Xian-hua Zeng. A random subspace method for co-training. In *2008 IEEE International joint conference on neural networks (IEEE World Congress on Computational Intelligence)*, pages 195–200. IEEE, 2008.

[31] Wei Wang and Zhi-Hua Zhou. Analyzing co-training style algorithms. In *European conference on machine learning*, pages 454–465. Springer, 2007.

[32] David Yarowsky. Unsupervised word sense disambiguation rivaling supervised methods. In *33rd annual meeting of the association for computational linguistics*, pages 189–196, 1995.

[33] Zhi-Hua Zhou. *Ensemble methods: foundations and algorithms*. Chapman and Hall/CRC, 2025.

[34] Zhi-Hua Zhou and Ming Li. Semi-supervised learning by disagreement. *Knowledge and Information Systems*, 24(3):415–439, 2010.

[35] Xiaojin Zhu and Andrew Goldberg. *Introduction to semi-supervised learning*. Morgan & Claypool Publishers, 2009.

[36] Xiaojin Jerry Zhu. Semi-supervised learning literature survey. 2005.